

LLM 推論與服務容量

一個 request 進到 GPU 之後發生什麼事，以及「能服務多少人」該怎麼算

- Q1 模型的一次 forward，錢花在算力還是頻寬上？
- Q2 混合架構把 KV cache 換成固定大小的狀態，記憶體帳本長什麼樣？
- Q3 NVFP4 與 FP8 到底量化了哪些張量，省下多少？
- Q4 speculative decoding 什麼時候賺、什麼時候該關掉？
- Q5 從實測數字到「可以承諾幾個人用」，中間要補哪些前提？

怎麼讀這份報告

先講清楚顏色、名詞與數字的可信度分級

顏色一律代表同一件事

- 算力 / attention / 權重
- 記憶體頻寬 / KV cache
- GDN、SSM 狀態
- MoE / expert
- speculative decoding
- 被接受 / 有效產出
- 被拒絕 / 浪費

看不懂的字直接把滑鼠放上去

像 PagedAttention、Gated DeltaNet、接受長度 τ 這種虛線底的粗體字 都是專有名詞，滑過會出現解釋，按 **G** 可以打開完整名詞表（共 41 條）。

每一頁的頁尾都列出該頁的來源，可直接點開原始 config、論文或官方文件；按 **N** 的講者筆記也會附同一組連結。

數字的可信度分三級

- 實測 / 官方** 直接來自官方 config、模型卡、checkpoint 的 safetensors 位元組統計，或論文與廠商公布的 benchmark。可以直接引用。
- 公式推導** 由上面那些數字用明確公式算出來的，例如 KV cache 大小、記憶體帳本、**roofline** 上限。公式都寫在頁面上，可以自己驗算。標示為「解析上限」的，實測通常只有它的 30–60%。
- 教學示例** throughput / 延遲 / 佇列曲線與 SLO 掃描示意圖。用排隊理論生成，**不是任何硬體的實測結果**，只用來說明形狀。

凡是標成教學示例的圖，都不可以當成 B200 實測成績對外引用。正式容量報告必須換成自家實測資料。

報告路線

從一個 token 的產生，一路推到「能服務幾個人」

01	一個 token 是怎麼生出來的	autoregressive、KV cache、GQA、prefill vs decode，最後導出 B200 的屋頂線轉折點	8 分
02	vLLM 把它變成一個服務	執行堆疊、continuous batching、PagedAttention、每一輪排程實際在做什麼	7 分
03	三個模型的架構帳本	純 Transformer、hybrid dense、hybrid MoE；參數如何逐張量驗證	10 分
04	Gated DeltaNet：用固定狀態換掉 KV cache	state update 的數學、hybrid 的 3:1 排列、vLLM 怎麼同時管兩種 cache	10 分
05	量化：NVFP4 與 FP8	位元佈局、checkpoint 實際量化了哪些張量、精度代價、為什麼轉折點不會移動	8 分
06	單卡記憶體帳本	KV cache 與 SSM state 分開算、交會點、B200 180 GB 怎麼分、併發上限	10 分
07	Speculative decoding 對 serving 的影響	MTP 與 DFlash2 的機制差異、接受長度、(k+1)× 撞屋頂線、什麼時候該關掉	12 分
08	延遲指標的定義與陷阱	TTFT / TPOT / ITL / E2EL、百分位、goodput	6 分
09	兩階段負載測試	閉環找飽和、開環找 SLO 容量、邊界搜尋、RPS 換算人數	12 分
10	結論與調校決策樹	十個可以帶走的結論，加一張「該轉哪個旋鈕」的流程圖	5 分

一個 token 是怎麼生出來的

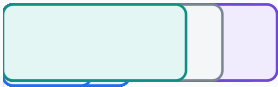
推論的成本結構就是在這一段裡定下來的。這一章結束時會得到一個數字，後面所有現象都可以拿它來解釋：B200 上的屋頂線轉折點。

- tokenizer 與 chat template：模型真正看到的長度
- autoregressive 迴圈：為什麼生 T 個 token 要 T 次 forward
- decoder layer 的內部與張量形狀
- KV cache 做了什麼、GQA 又省下多少
- prefill 與 decode 是兩種完全不同的工作
- 算術強度與 roofline： $N^* \approx 292 \text{ token / step}$

模型真正看到的是一串 id

用官方 tokenizer 實際跑一次，id 都是真的

13 個中文字切成 8 個 token，加上 chat template 之後大約 20 個。容量計算的分母是這個數字，不是字數。



量子電腦的錯誤修正是什麼？

13 個中文字

② tokenizer 切成 token，再查表換成整數 id

→ 8 個 token

113672 110335 95726 116235 96491 100643 98900 10992

注意「修正」被切成「修」+「正是」。BPE 只看統計上常一起出現的位元組，不管詞的邊界。

③ chat template 再包上角色標記，這才是真正送進模型的東西

248045

248046

248045

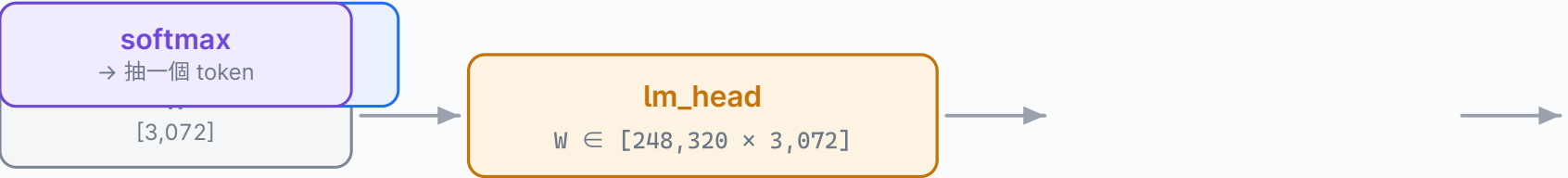
④ 這串 id 的長度 T 才是「context 長度」。13 個字的問句，實際送進去大約 20 個 token。
整份報告裡 KV cache 的公式、記憶體帳本、prefill 成本，分母都是這個 T，不是字數。

③ 這串 id 的長度 T，才是報告裡所有公式的分母。

Vocabulary 與 lm_head

為什麼那張 $248,320 \times \text{hidden}$ 的表，每一步都要整個讀完

embedding 是「用 id 找一列」，lm_head 是「對每一列都算一次內積」。同樣的形狀，成本差了幾千倍。



$\text{logits}[i] = \sum h[j] \cdot W[i][j] \quad , i = 0 \dots 248,319$
要算出第 i 個分數，就要讀 W 的第 i 列。而 softmax 要對全部 248,320 個分數做正規化，一個都不能少。

embed_tokens 同樣是 $[V \times d]$ ，但只讀 1 列
查表：拿到 token id 就取出那一列，讀 $3,072 \times 2 \text{ B} = 6 \text{ KiB}$ 。幾乎免費。

lm_head 整個 $[V \times d]$ 都要讀
 $248,320 \times 3,072 \times 2 \text{ B} = 1.53 \text{ GB}$ ，每個 decode step 一次。

差別在於：embedding 是「用 id 找列」，lm_head 是「對每一列都算一次內積」。
batch B 時這 1.53 GB 只讀一次、被 B 個 token 分攤，又是同一個屋頂線的故事。

batch B 的時候這 1.53 GB 只讀一次、被 B 個 token 分攤，又回到同一個屋頂線的故事。

Autoregressive：生 T 個 token 就要 T 次 forward

decode 之所以慢，原因就在這裡

模型一次 forward 只能決定「下一個」token。整個 serving 最佳化史，就是想辦法讓每次 forward 更划算。



推論不做的事

沒有 backward、沒有 optimizer state。權重固定不動，變動的只有 activation 與每個 request 的狀態。

什麼時候停

抽到 EOS、命中 stop string、或達到 max_tokens。輸出長度事前不知道，這讓容量規劃必須用分布而非單一值。

有一個例外

Speculative decoding 可以讓一次 forward 前進多格。那是 CH7 的主題，機制上還是同一個迴圈。

Decoder layer 內部：兩個 mixer

token mixer 讓位置之間交換資訊，channel mixer 只在單一位置上做事

每一層都是「先跨位置混一次，再跨通道混一次」。attention 與 GDN 是可以互換的 token mixer；FFN 與 MoE 是可以互換的 channel mixer。

The diagram illustrates the internal structure of a decoder layer. It starts with a 'token mixer' stage, which includes a box labeled 'FFN / MoE' with the note '每個 token 各自算' (each token calculated individually). This is followed by a 'channel mixer' stage. A 'residual' connection is shown as a dashed green line that bypasses the main processing and is added to the output. A '穩定數值' (stable value) is also indicated in the token mixer stage.

資料形狀 (Qwen3.5-122B-A10B，decode，batch B，每序列 1 個 token)

hidden	[B, 3072]
Q	[B, 32 heads, 256] 總寬 8192
K, V	[B, 2 heads, 256] ← 只有 2 組
RoPE 旋轉範圍	只有 head_dim 的 1/4 = 64 維
讀取 KV cache	[B, 2, T, 256] × 2 T = 目前長度
attention 輸出	[B, 32, 256] → o_proj → [B, 3072]
MoE	256 個 expert 只算 top-8 + 1 shared

KV cache 的大小只跟 K/V head 數與 head_dim 有關，跟 Q 的 32 組無關。所以「Q 很寬」不花記憶體，只花算力。GQA 的好處就在這裡。

Channel mixer (FFN 或 MoE) 每個 token 各自算，位置之間互不相干。

兩種 mixer 各自解決一個問題

「去哪裡拿資訊」與「拿到之後算出什麼」

Attention 負責通訊，FFN 負責運算。只有前者需要記憶體保存歷史，所以 cache 的形狀完全由它決定。

TOKEN MIXER

「我要去哪裡拿資訊？」

跨位置搬運。只有它需要記憶體保存歷史。

Attention · Gated DeltaNet · SSM

→ KV cache / state cache



CHANNEL MIXER

「拿到之後要算出什麼？」

同一個位置內，把所有 feature 重新組合成新的 feature。

FFN / SwiGLU · MoE

→ 不需要任何歷史 cache

舉例：The cat didn't eat the fish because it was sick.

- Attention 把資訊搬過來

「it」 這個位置從前面拿到：cat 是 animate、是前句主詞、sick 與 eat 有否定關係。
- FFN 把它們組合成新的東西

animate + 主詞 + sick 這幾個 feature 一起出現 → 「it 指的大概是 cat」。
- 所以有人用這組對照來記

Attention = communication（通訊），FFN = computation（運算）。

換掉 token mixer，cache 的形狀就變了；換掉 channel mixer，只有算力與權重大小變。Qwen 的混合架構動的就是前者。

換掉 token mixer，cache 的形狀就變了；換掉 channel mixer，只有算力與權重大小變。

Causal mask：為什麼舊 token 不用重算

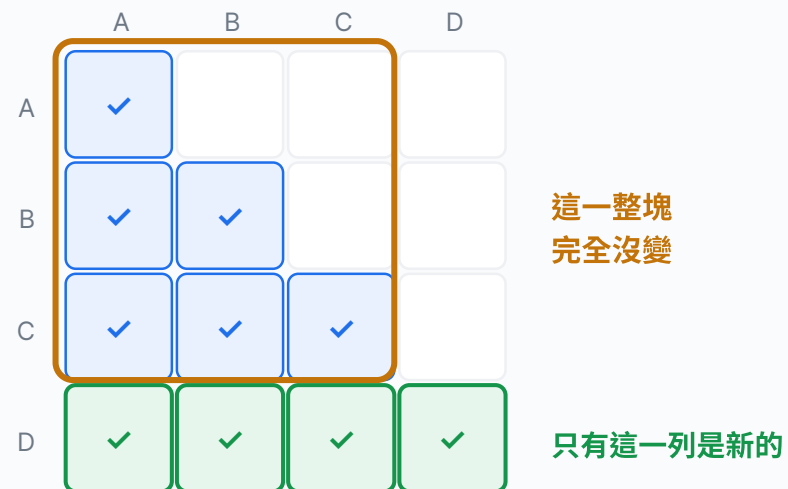
KV cache 之所以成立的前提

未來的 token 不會改變過去 token 已經算好的 K/V。所以序列雖然每次 +1，但只有最後一列是新的。

Prefill 完 A B C 之後，causal attention 是一個下三角



生出 D 之後，矩陣多一行一列



為什麼沒變

A、B、C 不可能突然看得到 D。causal mask 保證未來不會影響過去，所以它們算好的 K/V 永遠有效。

所以只要算新的一列

$Q_D \times [K_A, K_B, K_C, K_D]$ 。前面三列一個都不用重算。

KV cache 就是幹這個的

把 prefill 算好的 K/V 存起來，decode 時只 append 新 token 的 K_D 、 V_D 。

但它沒有解決另一件事

Q_D 還是得掃過全部 T 個位置的 K/V。所以 decode 的 attention 成本是 $O(T)$ ，context 越長越貴。

KV cache 解決的是「不要重算歷史」。它沒有解決「現在的 query 還是得掃過全部歷史」。

為什麼是 KV cache，不是 QKV cache

Q 是一次性的問題，K/V 才是留給未來查的東西

Q 問完就沒用了，歷史的 Q 永遠不會再被查詢；K 是可被查詢的索引，V 是查到之後要讀出來的內容。



Q 是「我現在想查什麼」，一次性的問題；K/V 是「我留給未來查的東西」。所以只有 K 和 V 需要 cache。

Attention：append

舊的 K/V 一個位元組都不動，
只在尾巴多一格。memory $\propto T$ 。

GDN：就地覆寫

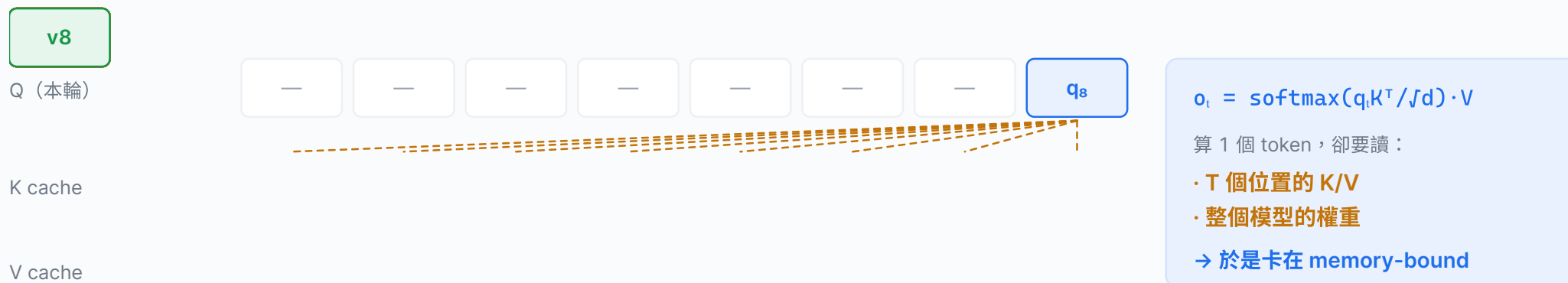
每個 token 都把同一份固定大小的
state 改掉。memory 是常數。

順帶把兩種 cache 的差別擺在一起：attention 是 append，GDN 是就地覆寫。

KV cache：把算過的東西存起來

代價是記憶體隨 context 線性成長

沒有 **KV cache**，生成成本是 $O(T^2)$ ；有了 KV cache 變成 $O(T)$ ，但要付出隨 T 成長的記憶體。



沒有 cache：每一輪都要把前面 $T-1$ 個位置重算，總成本 $O(T^2)$

④ 若不快取，每輪都要把前面所有位置重算一次，總成本變成 $O(T^2)$ 。

換來的是什麼

計算從 $O(T^2)$ 降到 $O(T)$ 。長 context 之下這是不可能放棄的。

付出的是什麼

每個 token 都要永久佔一塊 **HBM**，直到 request 結束。這是併發數的**硬上限**。

三種解法

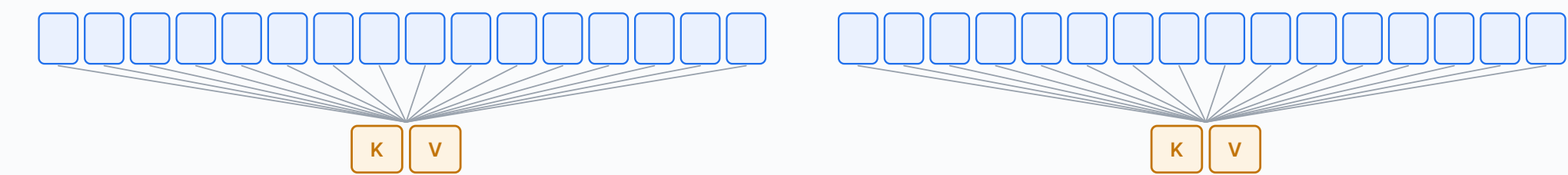
① **GQA** 減少 KV head（本頁下一張）② **FP8** KV cache 砍一半 ③ 換掉 token mixer，讓它不需要 KV，也就是 CH4 的 **GDN**。

GQA：Q 很多、K/V 很少

KV cache 的大小只跟 KV head 數有關，跟 Q head 數無關

122B 用 32 個 Q head 共用 2 組 K/V，等於把 KV cache 壓成 MHA 的 1/16。這是設定檔裡最便宜的一個決定。

Qwen3.5-122B-A10B full-attention 層：32 個 Q head 共用 2 組 K/V head (head_dim 256)



→ 每個 token 只需要存 $2 \times 2 \times 256 = 1,024$ 個數字 (K 與 V)

所以每個 token 只需要保存 $2 \times 2 \times 256 = 1,024$ 個數字 (K 與 V 各 512)。

模型	Q HEAD	KV HEAD	HEAD_DIM	共用比例	假設 MHA 的 KV/TOKEN	實際 KV/TOKEN (BF16)
Qwen3.5-122B-A10B	32	2	256	16:1	384 KiB	24 KiB
Qwen3.8-27B	24	4	256	6:1	384 KiB	64 KiB
Qwen3-32B (純 Transformer)	64	8	128	8:1	2,048 KiB	256 KiB

「假設 MHA」欄是把 KV head 數換成 Q head 數重算的結果，用來看 GQA 省了多少。122B 的 KV/token 之所以特別小，是 GQA (16:1) 與只有 12 層 full attention 兩件事相乘的結果。

Prefill 與 decode：兩種完全不同的工作

同樣的權重、同樣的層，成本結構卻相反

prefill 是「很多 token 分攤一次權重讀取」，decode 是「很少 token 分攤一次權重讀取」。前者吃算力，後者吃頻寬。

PREFILL 一次吃完整個 prompt



矩陣運算： $[T, 3072] \times \text{權重} \rightarrow \text{大 GEMM}$

T 個 token 一起攤提同一份權重讀取

算術強度（每讀 1 byte 做幾次 FLOP）



DECODE 每輪每條序列只 1 個 token



矩陣運算退化成 GEMV（瘦長矩陣）

同一份權重只服務 B 個 token

算術強度



所有 serving 最佳化都在回答同一件事：
怎麼讓每次讀取權重的成本，被更多 token 分攤。continuous batching、chunked prefill、speculative decoding 都在做這件事。

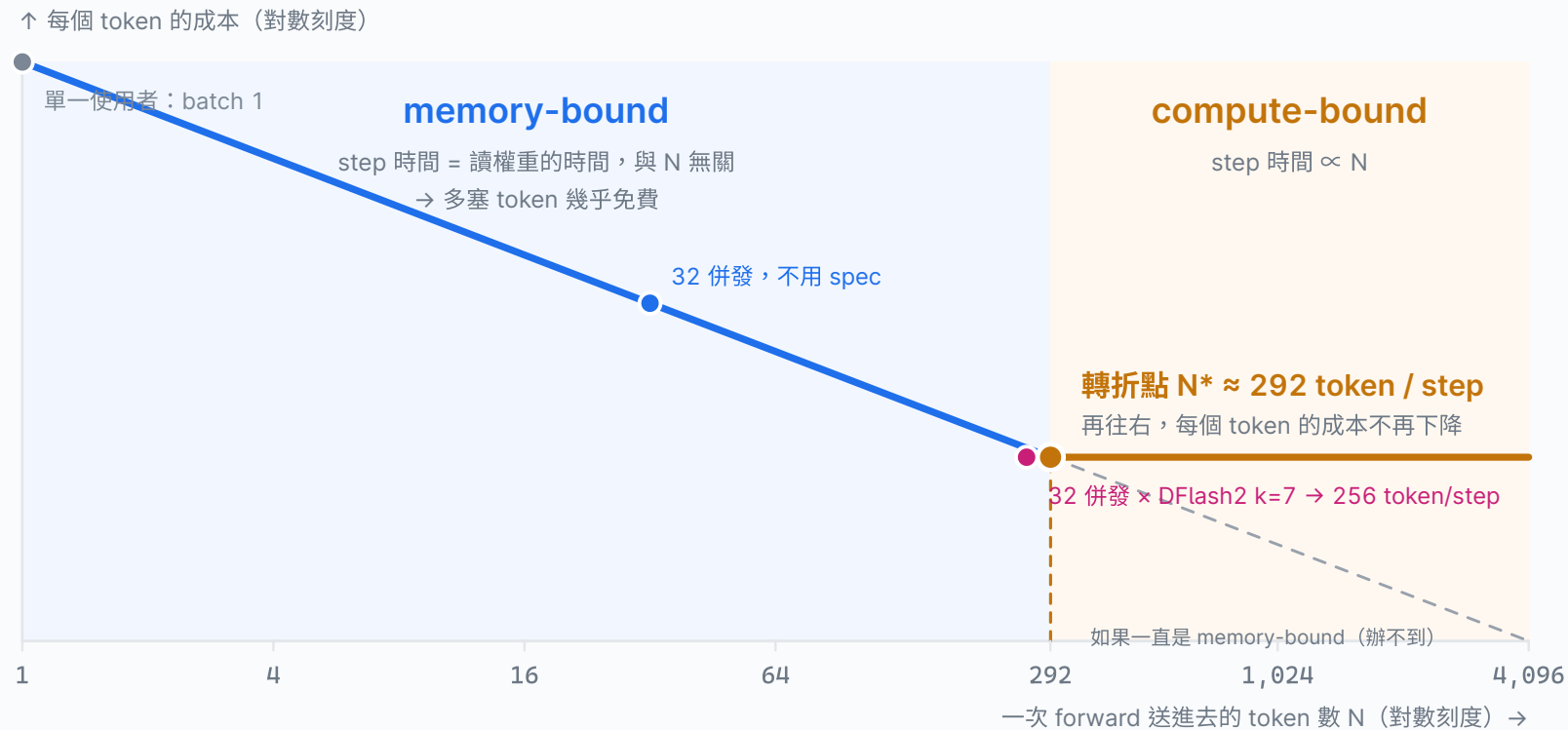
看清這件事之後，serving 上的最佳化就收斂成同一個動作：把更多 token 塞進同一次權重讀取裡。

	PREFILL	DECODE
本輪 token 數	prompt 全長（可達數萬）	batch 大小 B（每序列 1 個）
矩陣運算	大 GEMM，Tensor Core 打滿	GEMV／瘦 GEMM，Tensor Core 大量閒置
瓶頸	算力	記憶體頻寬
attention 成本	$O(T^2)$ ，但可用 FlashAttention 分塊	$O(T)$ ，隨 context 線性上升
對指標的影響	決定 <u>TTFT</u>	決定 <u>TPOT</u> 與 <u>ITL</u>
調校旋鈕	chunked prefill、prefix caching	batch 大小、量化、speculative decoding

屋頂線：B200 上的轉折點是 292 token / step

而且它不會因為量化而移動

一次 forward 裡湊到約 292 個 token 之前，時間都花在讀權重上、加 token 幾乎免費；超過之後才開始付算力的錢。



$$N^* = F \cdot b / (2 \cdot BW)$$

F 峰值算力 (FLOP/s)

b 每個參數的位元組數

BW HBM 頻寬 (B/s)

Blackwell 每把位寬砍半，
張量核心吞吐就剛好加倍：
F 加倍、b 減半 $\rightarrow$ 相消。

精度	b	F (PF)	N^*
BF16	2	2.25	292
FP8	1	4.5	292
NVFP4	0.5	9.0	292

而 32 併發配上 [DFlash2](#) 的 7 個草稿，一次就送進 256 個 token，剛好貼在轉折點上。CH7 要算的就是這件事。

量化幫不上這個忙

FP8 或 NVFP4 讓兩邊**同時**變快，你在**屋頂線**上的相對位置完全沒動。「上了 FP4 就不再 memory-bound」是很常見的誤會。

想離開只有一條路

要離開 memory-bound 只有一條路：**提高每個 step 的 token 數**：更大 batch，或 [Speculative decoding](#)。

反過來說

只要 $N < 292$ ，就有**免費的算力**可以拿去猜 token。
speculative decoding 之所以成立，靠的就是這塊。

N 指的是哪 292 個 token

這個數字最常被誤讀，先把它釘死

N 是「這一次 forward 裡共用同一批權重做 GEMM 的 token 位置數」。不是 context 長度，也不是「一次只能生 292 個 token」。

三個它不是的東西

- 不是 context 長度，也不是 `--max-model-len`
- 不是「B200 一次只能生 292 個 token」
- 不是每張卡的容量上限（那是 CH6 的 KV cache 帳）

decode 時 $N \approx B$

32 條併發就是 $N = 32$ 。就算每條的 context 都已經 100k，這一步對權重 GEMM 而言仍然只有 32 個 token 位置——長 context 增加的是 attention 讀 KV 的流量，**不在這條屋頂線裡**。

同一個 N，三種來源

情境	N 是什麼	量級
decode（無 spec）	併發數 B	1 – 60
decode + spec	$B \times (k+1)$	$B \times 8$
prefill（分塊）	本輪排進去的 prompt token	→ 16,384

N（這一步的 TOKEN 位置數）	搬 27B 的 FP8 權重	張量運算	一個 STEP 要多久
1	3.50 ms	0.012 ms	3.50 ms
32	3.50 ms	0.383 ms	3.50 ms
128	3.50 ms	1.531 ms	3.50 ms
256	3.50 ms	3.063 ms	3.50 ms
292	3.50 ms	3.493 ms	3.50 ms
512	3.50 ms	6.125 ms	6.13 ms

「292 以下幾乎免費」不是說 token 沒有成本，而是**那 3.5 ms 的權重搬運費你本來就要付**，而張量核心在這段時間裡幾乎閒著（ $N=32$ 時只用掉約 11%）。這是理想化的 roofline，不含 attention 讀 KV、norm、量化 scale 與排程。

vLLM 把它變成一個服務

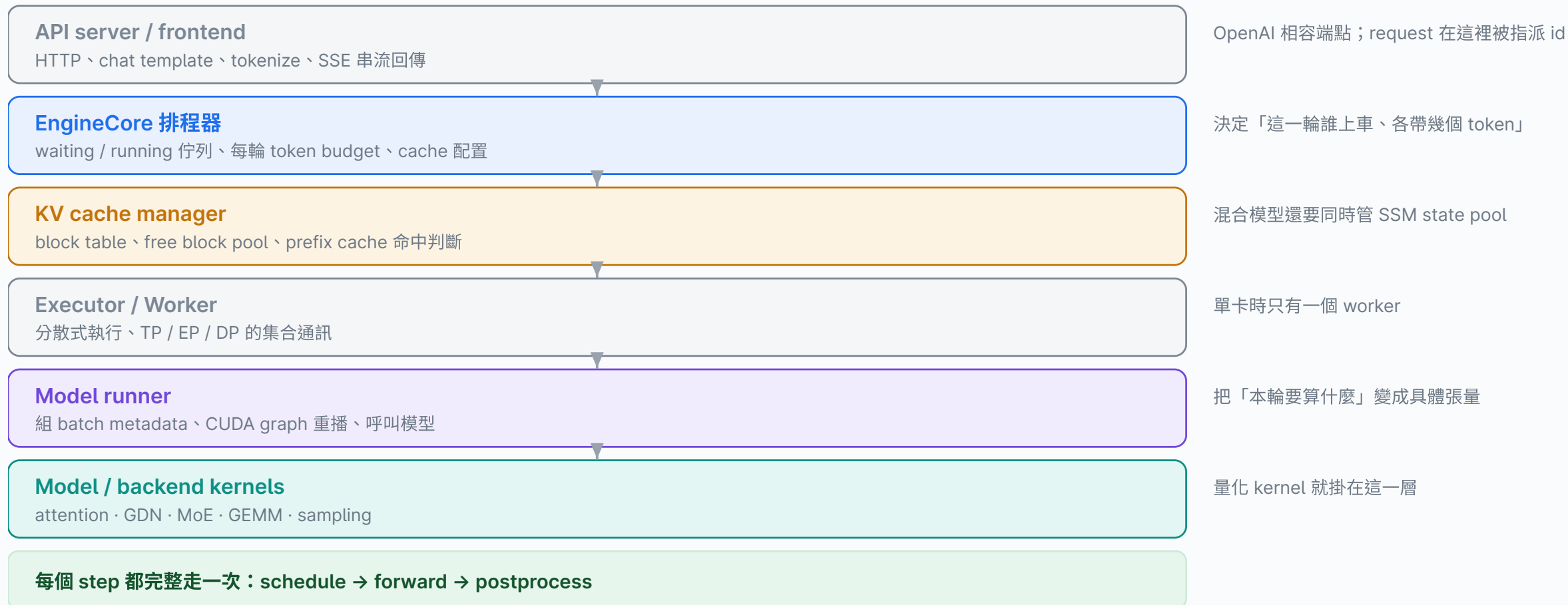
模型只回答「這一個 token 是什麼」。要變成能同時服務數百人的服務，中間這一層要解決排程、記憶體分頁與批次組合三個問題。

- 從 HTTP 到 GPU kernel 的責任邊界
- Continuous batching：每一輪重新組隊
- PagedAttention：把 KV cache 當成作業系統的分頁
- Scheduler 一輪實際在做的四個決定
- Chunked prefill 與 prefix caching
- 換模型的時候，哪一層真的會動

vLLM 的執行堆疊

六層責任邊界，換模型時只有最下面兩層需要改

上面三層是「服務」，下面三層是「模型」。看懂這條界線，就知道每個 CLI 旗標會影響什麼。



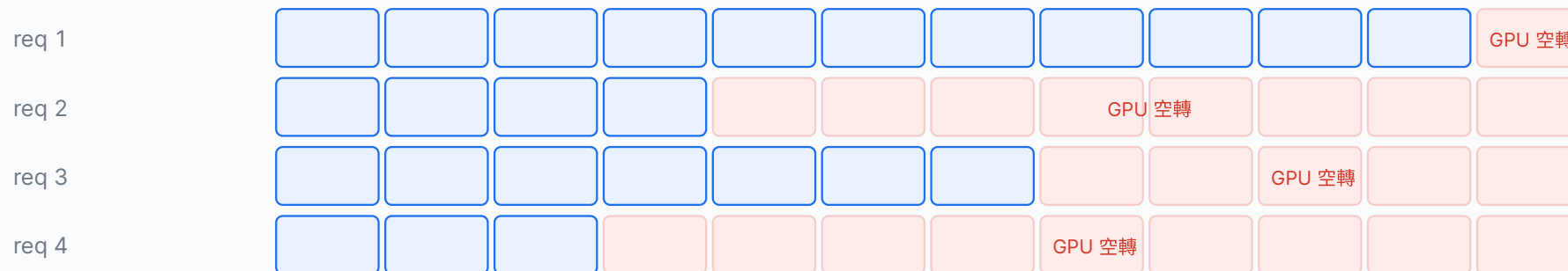
Kernels 才是真正動手的地方，量化 kernel 就掛在這一層。

Continuous batching：空出來的位置立刻補上

同一次權重讀取被更多 token 分攤

靜態 batching 要等最慢的那個人講完；**continuous batching** 每一輪重新組隊，所以 GPU 幾乎不空轉。

靜態 batching：整批一起開始、一起結束



continuous batching：每一輪重新組隊



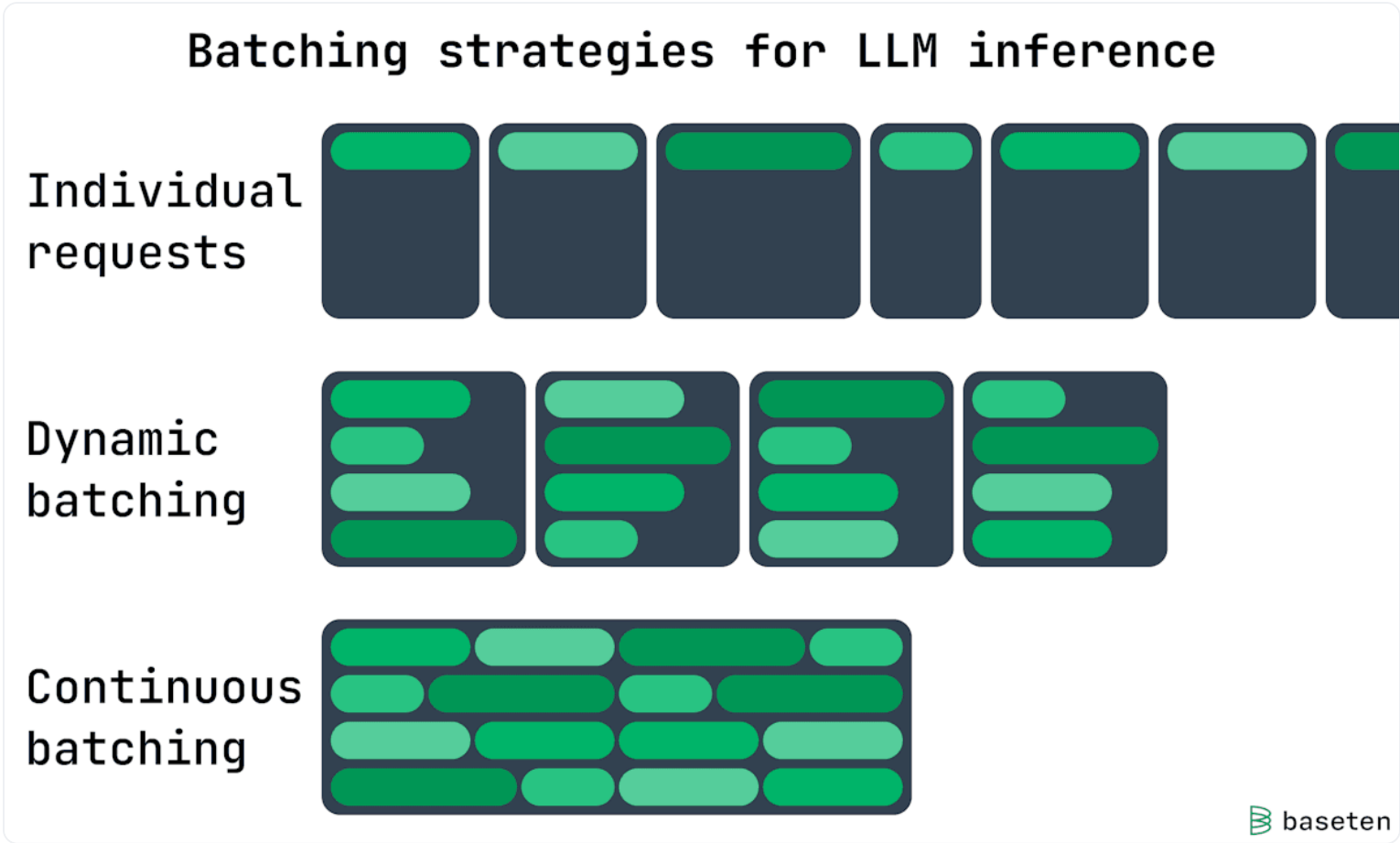
空出來的位置立刻被新 request 補上。同一次權重讀取被更多 token 分攤，就是 CH1 講的那件事。

所以同一份權重讀取被更多 token 分攤，右移到屋頂線上更划算的位置。

下一頁附上 vLLM 官方部落格的原始示意圖作對照。

原始示意圖對照

vLLM 官方部落格的 batching 策略圖



三種策略的差別

Individual	一次一個 request。權重讀取沒有分攤，最浪費。
Dynamic	湊滿一批才送，整批一起結束。有分攤，但尾端空轉。
Continuous	每輪重組。分攤最大化，GPU 幾乎不空轉。

這張圖與前一頁的自製動畫講的是同一件事。放在一起是為了讓你之後看到官方文件時能對得上。

圖片由使用者提供，出自 [vLLM 官方部落格](#) 《Inside vLLM》。三種策略由上到下：individual requests、dynamic batching、continuous batching。

PagedAttention：把 KV cache 當成作業系統的分頁

邏輯連續、實體不連續，碎片幾乎消失

KV cache 切成固定大小的 block，用 block table 對應。碎片沒了，前綴還能共用。

free

request 是一條連續的 token 序列

req A

The

future

of

AI

is

req B

Paged

memory

avoids

req C

Blocks

map

實體上：切成固定大小的 block，散落在 HBM 各處

block table 記住「邏輯第幾塊 → 實體第幾塊」

req A → [0, 4, 8]

req B → [3, 6]

req C → [1, 10]

得到三件事：

① 幾乎沒有碎片，只浪費最後一塊的尾巴

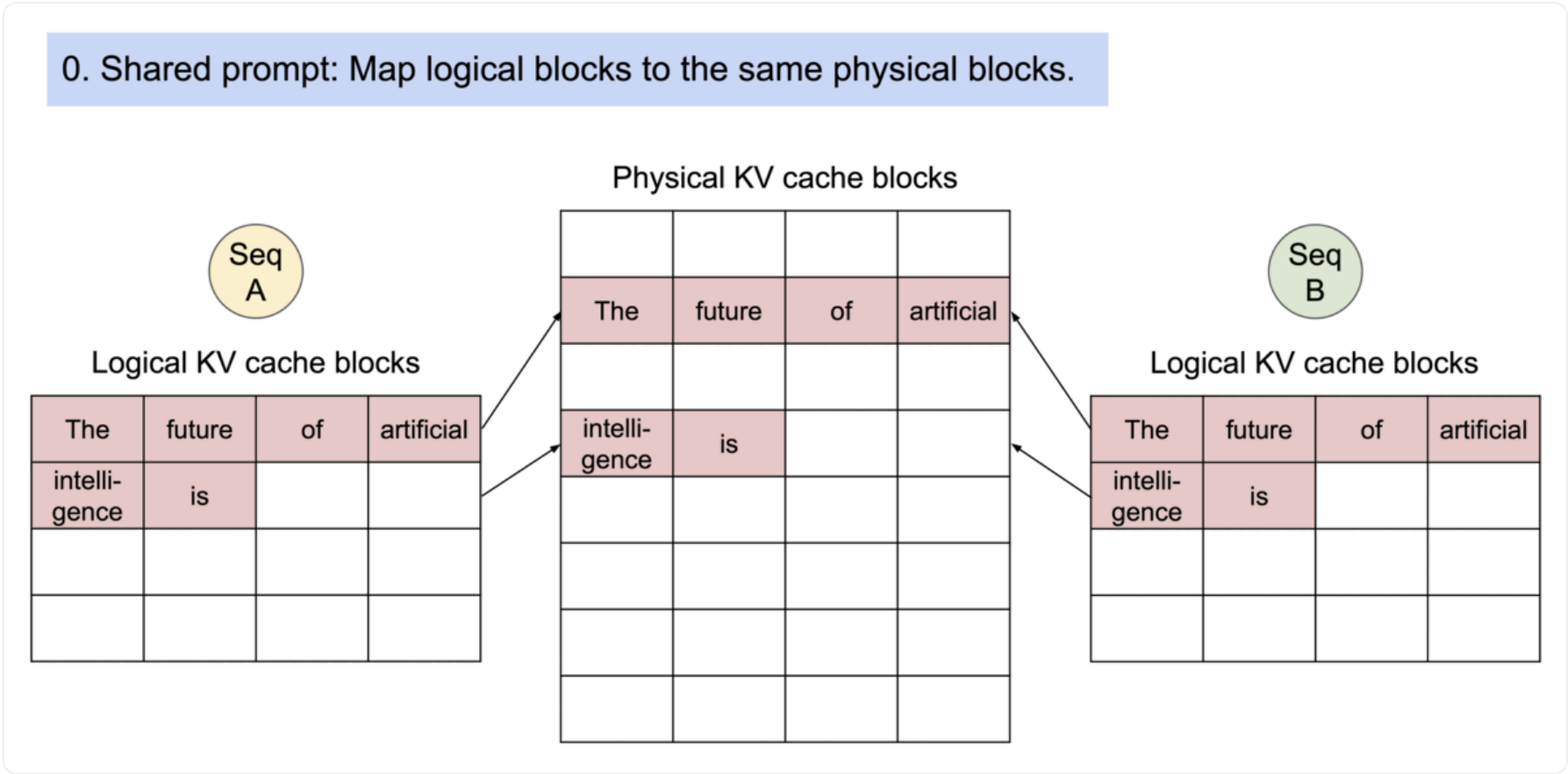
② 相同前綴可以共用同一塊（prefix caching / 分支）

③ request 可以被 preempt 換出，之後再換回來

換來三個能力：沒有碎片、前綴可共用、request 可以被換出再換回。

原始示意圖對照

vLLM 官方部落格的 PagedAttention 動畫



圖片由使用者提供，出自 [vLLM PagedAttention](#) 發表文。動畫展示兩個共用同一段 prompt 的 request，如何對應到相同的 physical KV block。

看這張圖時注意兩件事

- 兩個 request 的 logical block 0 指向**同一個** physical block，**prefix caching** 就是靠這個做的。
- Logical 與 physical 的順序完全無關；順序只存在 block table 裡。

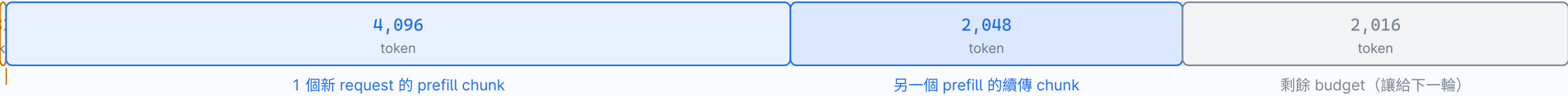
這張是 GIF，會自動播放，與自製動畫的「下一步」按鈕分開。

Scheduler 一輪做的四個決定

token budget 怎麼被切開，決定了 TTFT 與 TPOT 的取捨

排程器先保住正在生成的人的 decode，再用剩下的 budget 去餵新來的 prefill。這個順序就是 TPOT 優先於 TTFT。

本輪 token budget = max_num_batched_tokens = 8,192



32 個 decode request × 1 token
排程器同時看兩個佇列與兩種資源

running 佇列

已在生成中；優先保住它們的 decode

waiting 佇列

還沒開始；FCFS 或 priority

KV / SSM cache 餘量

配不到 slot 就得 preempt 別人

token budget

本輪算力與頻寬的預算上限

決策順序：① 先排 running 的 decode (保 TPOT) ② 用剩下的 budget 排 prefill chunk (追 TTFT)
③ 若 cache 不足，從尾端 preempt request 換出 ④ 產生 batch metadata，交給 model runner

④ 最後把 batch metadata 交給 model runner。

為什麼 DECODE 優先
已經在串流的使用者如果卡住，體感是「打字停住」，比多等一點 TTFT 難忍受得多。

BUDGET 開大開小
開大 → prefill 吃更多，TTFT 好、正在生成的人 ITL 變抖。開小 → 反之。這是最直接的 TTFT/TPOT 旋鈕。

PREEMPT 的代價
換出的 request 之後要重算或搬回，對混合模型還要一併處理 SSM state。頻繁 preempt 是容量過載的明顯訊號。

Chunked prefill 與 prefix caching

兩個最容易被低估的旗標

chunked prefill 讓長 prompt 不會卡住別人；prefix caching 讓共用開頭的 prompt 只算一次。

CHUNKED PREFILL

把長 prompt 的 prefill 切成小塊，分散到多個排程輪次。

- 沒有它：一個 128k 的 prompt 會獨占好幾個 step，同時在線的人 ITL 出現長尖峰。
- 有它：每一輪都能混入 decode，尖峰被抹平。
- 副作用：那個長 prompt 自己的 TTFT 會變長，這是刻意的取捨。
- 旗標：`--long-prefill-token-threshold` 與 budget 一起決定切多細。

PREFIX CACHING

不同 request 共用的開頭（system prompt、few-shot、對話歷史）只算一次，KV block 直接重用。

- 逐 block 做雜湊；命中就跳過那段 prefill。
- 對 agent／多輪對話效果最大，常見命中率 60–90%。
- 混合模型：full attention 由左往右找命中，兩種 group 取交集；vLLM 對 mamba / GDN 的 prefix caching **仍在開發中**。
- Speculative decoding 會干擾命中率，CH7 有實測案例。

「*Chunked prefill is a technique for handling long prompts by splitting their prefill step into smaller chunks.*」
Inside vLLM: Anatomy of a High-Throughput LLM Inference System, 2025-09-05

換模型的時候，哪一層真的會動？

分清「同架構換權重」與「換架構」的成本差距

同架構換 checkpoint 幾乎零成本；換 token mixer 就要動到 kernel 與 cache 管理，是完全不同的工程量。

層級	同架構、換 CHECKPOINT（例如換量化版）	換架構（例如 TRANSFORMER → 混合）
REST API	不動	不動；只有新的模態或輸出解析器才需要
Tokenizer / chat template	多半沿用，但要核對 <code>tokenizer_config</code> 與模板	多半沿用（Qwen 系列 vocab 相同）
模型載入 / 權重對應	要對應新的 tensor 名稱與量化 metadata	要新的模型類別與並行切法
排程器	不動	不動（介面相同）
Cache 管理	只變大小（KV dtype、層數）	要新增一種 cache：SSM state pool、page size 對齊、prefix caching 規則
GPU kernels	要有對應的量化 kernel（例如 <code>modelopt_fp4</code> ）	要新的 token-mixer kernel（chunkwise GDN、state update）
<u>Speculative decoding</u>	起草器要與新 checkpoint 相容（vocab、hidden size）	起草器也要能處理新的 cache 形狀

實務結論：從 Qwen3.8-27B 換到 Qwen3.8-27B-FP8 是設定層面的事；從 Qwen3-32B 換到 Qwen3.5-122B-A10B 則需要引擎本身支援 GDN 與 hybrid cache manager。評估新模型時，先確認 vLLM 版本支不支援，再談效能。

三個模型的架構帳本

同一個 decoder 骨架，換掉兩個插槽就得到三種完全不同的成本結構。這一章的每個數字都會回頭跟 checkpoint 的實際位元組數對帳。

- Qwen3-32B：純 Transformer 基準線
- Qwen3.8-27B：混合 token mixer + dense FFN
- Qwen3.5-122B-A10B：混合 token mixer + **MoE**
- MoE 的算力與記憶體錯位：A10B 到底是什麼意思
- 參數帳本：我們怎麼驗證這些數字沒有算錯

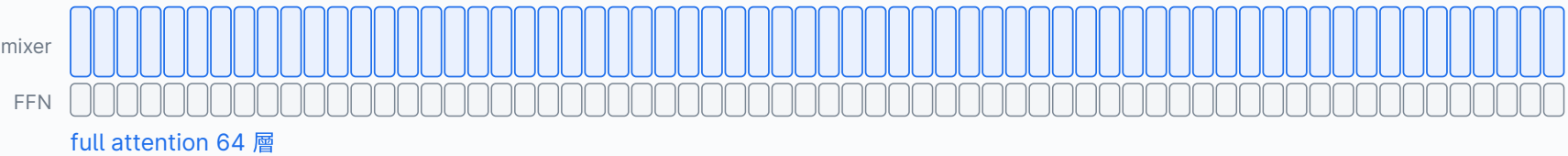
三個模型並排

換掉 token mixer 與 channel mixer 兩個插槽

同一個骨架，三種取捨：32B 全部 full attention、27B 四層裡放一層、122B 也是四層放一層但 FFN 換成 MoE。

Qwen3-32B

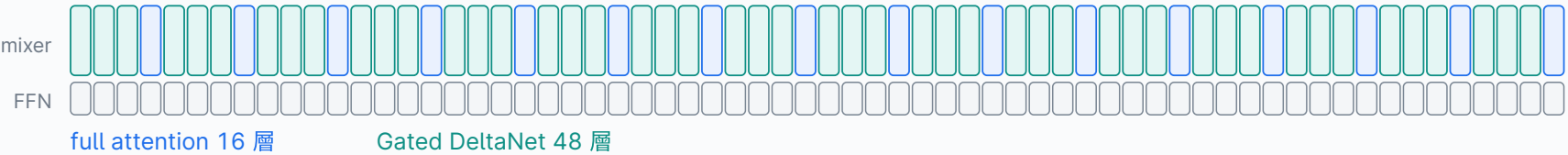
64 層 · hidden 5,120 · Dense FFN
BF16 · 65.5 GB



KV 256 KiB/tok
無 SSM state

Qwen3.8-27B

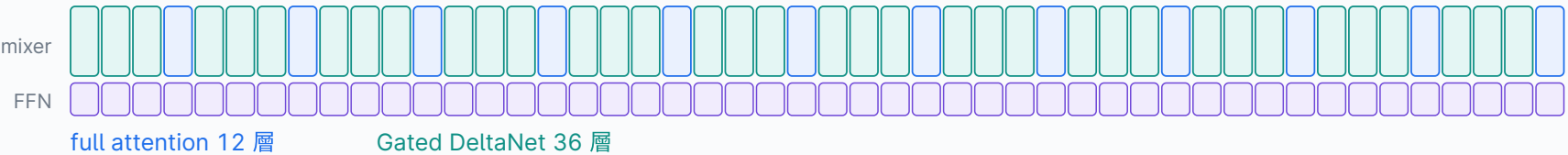
64 層 · hidden 5,120 · Dense FFN
FP8 · 30.9 GB



KV 64 KiB/tok
state 147 MiB/序列

Qwen3.5-122B-A10B

48 層 · hidden 3,072 · MoE
NVFP4 · 83.5 GB



KV 24 KiB/tok
state 147 MiB/序列

顏色： ☐ full attention (要 KV cache) ☐ Gated DeltaNet (固定大小 state) ☐ MoE FFN ☐ Dense FFN

所以 KV/token 差了一個數量級。後面的記憶體帳本都從這裡長出來。

Qwen3-32B：純 Transformer 基準線

所有比較的原點

64 層全部 full attention。每個 token 要付 256 KiB 的 KV cache，比 122B 高 11 倍。

項目	數值	說明
層數	64	全部 full attention，沒有 linear attention
hidden size	5,120	
Q / KV head	64 / 8	<u>GQA</u> 8:1
head_dim	128	比 Qwen3.5 系列的 256 小一半
FFN intermediate	25,600	Dense SwiGLU
原生 context	40,960	config 的 max_position_embeddings
總參數	32.76 B	解析值與 checkpoint 完全一致
<u>BF16</u> 權重	65.52 GB	safetensors 實際位元組數

256KiB

每個 token 的 KV cache (BF16)

10GiB

單條序列跑滿 40k context

為什麼它的 KV 這麼貴

$2 \times 64 \text{ 層} \times 8 \text{ KV head} \times 128 \text{ dim} \times 2 \text{ B} = 256 \text{ KiB}$

關鍵是層數：64 層每一層都要存。混合架構把這個 64 降成 12 或 16，就直接把 KV 砍掉 3/4。

在 B200 上，光是 40k context 的一條序列就要 10 GiB。扣掉 61.02 GiB 的權重後，單卡大約只放得下 8 條。

Qwen3.8-27B：混合 token mixer + dense FFN

本次要上線的 FP8 版本

64 層裡有 48 層 GDN、只有 16 層 full attention。KV/token 因此降到 64 KiB，但多了固定 146.8 MiB 的 SSM state。

項目	數值	說明
層數	64	16 × (3 × GDN → 1 × full attention)
hidden size	5,120	
full attention	16 層	Q 24 / KV 4，head_dim 256
Gated DeltaNet	48 層	value head 48、key head 16、head dim 128
FFN	17,408	Dense SwiGLU (不是 <u>MoE</u>)
<u>MTP</u> head	1 層	checkpoint 內含 <code>mtp.*</code> ，可直接做 spec decode
原生 context	262,144	YaRN 可外推到約 1 M
總參數	27.79 B	語言 26.90 B + MTP 0.42 B + 視覺 0.47 B
<u>FP8 權重</u>	30.87 GB	= 28.75 GiB (BF16 版是 55.56 GB)

64 KiB
KV / token (BF16)

147 MiB
SSM state / 序列 (固定)

兩條公式

$$KV = 2 \times 16 \times 4 \times 256 \times 2 \text{ B} = 64 \text{ KiB} / \text{token}$$

$$\text{state} = 48 \times 48 \times 128 \times 128 \times 4 \text{ B} = 144 \text{ MiB} / \text{序列}$$

上面那條隨 context 長大，下面那條**永遠不變**。CH6 會把兩者放在同一張帳本上。

這一版與第一版的差別

第一版報告用 BF16 假設，權重算 55.56 GB。改成官方 FP8 版之後降到 **30.87 GB**，省下 24.70 GB，這些空間全部可以拿去放 cache。

Qwen3.5-122B-A10B：混合 token mixer + MoE

本次要上線的 NVFP4 版本

122B 總參數、每個 token 只算 9.0B。KV/token 只有 12 KiB (FP8)，是三個模型裡最省的。

項目	數值	說明
層數	48	12 × (3 × (<u>GDN</u> → <u>MoE</u>) → 1 × (Attention→MoE))
hidden size	3,072	比 27B 小，但層數少、寬度靠 MoE 補
<u>full attention</u>	12 層	Q 32 / KV 2 (<u>GQA</u> 16:1)，head_dim 256
Gated DeltaNet	36 層	value head 64、key head 16、head dim 128
MoE	256 experts	每 token 選 8 個 + 1 shared，intermediate 1,024
<u>MTP head</u>	1 層	2.52 B 參數 (自己也是一層 MoE)
原生 context	262,144	YaRN 可外推到約 1 M
總參數	125.07 B	語言 122.11 B ← 型號裡的 122B
每 token 啟用	9.01 B	← 型號裡的 A10B
NVFP4 權重	83.47 GB	= 77.74 GiB (BF16 版是 250.17 GB)

12 KiB

KV / token (FP8 KV cache)

147 MiB

SSM state / 序列 (固定)

能塞進單張 B200 是重點

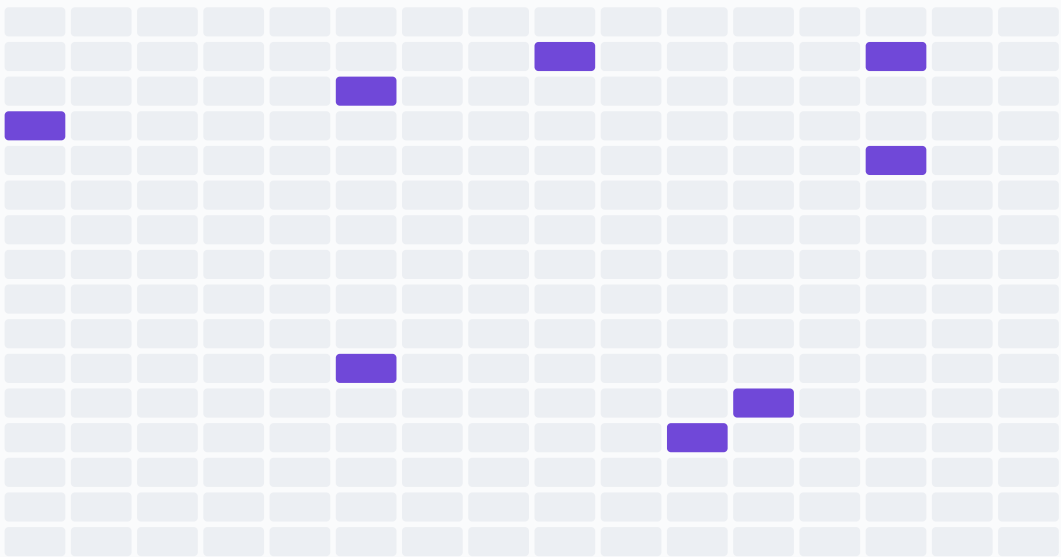
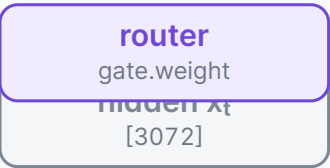
BF16 要 250.17 GB，一張 180 GB 的卡放不下，至少要兩張。NVFP4 之後只要 **83.47 GB**，官方模型卡的部署範例就是 `--tensor-parallel-size 1`。少了 TP 的 all-reduce，decode 的延遲與抖動都會明顯改善。

但別忘了 MTP head：2.52 B 參數在 NVFP4 checkpoint 裡**沒有被量化**，BF16 佔 5.05 GB。要做 speculative decoding 就得把這塊算進帳本。

MoE：算力與記憶體錯位

少算一些 expert，但整組權重都得放在卡上

MoE 用「稀疏算力」換「密集記憶體」。它讓模型變聰明而不變慢，但不會讓模型變小。



256 個 routed expert
每個 9.4 M 參數
本 token 只啟用其中 8 個
(深紫色) + 1 個 shared

算力：只算 8/256
每 token 啟用 9.01 B 參數
→ $2 \times 9.0B = 18 \text{ GFLOP} / \text{token}$
型號裡的 A10B 講的就是這個

記憶體：整組 256 個都得放在 HBM
 $48 \text{ 層} \times 256 \text{ expert} \times 9.4 \text{ M} = 116.0 \text{ B 參數}$
NVFP4 之下 = 65.2 GB
→ 佔了整個 checkpoint 的 69%

而且 batch 一大，被選到的 expert 就會涵蓋幾乎全部 256 個，記憶體流量從 2 GB 長到 65 GB。CH7 會用到這件事。

而且 batch 一大，被選到的 expert 幾乎涵蓋全部 256 個，記憶體流量從 2 GB 長到 65 GB。

參數帳本：我們怎麼確定沒算錯

解析公式 vs checkpoint 實際位元組數

每個模型的參數量都用 config 逐張量算一次，再與 HuggingFace 上 safetensors 的 dtype 統計對帳。誤差都在 0.02% 以內。

模型	解析推導參數	CHECKPOINT 實際參數	誤差	語言模型	MTP HEAD	視覺塔
Qwen3.5-122B-A10B	125,074,705,904	125,086,503,920	0.009%	122.11 B	2.52 B	0.439 B
Qwen3.8-27B	27,786,416,112	27,781,427,952	0.018%	26.90 B	0.42 B	0.466 B
Qwen3-32B	32,762,123,264	32,762,123,264	0.000%	32.76 B	—	—

逐層拆解（每一層的參數量）				
模型	GDN 層	ATTENTION 層	FFN / MOE 層	CONV_DIM
122B-A10B	88.5 M	78.6 M	2,426 M	12,288
27B	115.9 M	104.9 M	267.4 M	10,240
32B	—	94.4 M	393.2 M	—

為什麼要做這件事

因為後面每一張記憶體帳本、每一條 **roofline** 都建立在這些數字上。如果參數量算錯 10%，容量結論就會錯 10%。

對帳方式：把 config 裡的每個維度乘出來，逐個對應 checkpoint 的 tensor 名稱（`in_proj_qkv`、`mlp.experts.*`、`mtp.*` ...），再與 HF API 回報的 **BF16** / F8_E4M3 / U8 元素數比對。

CHAPTER 04

Gated DeltaNet

混合架構的核心。把「保存全部歷史 K/V」換成「維護一個固定大小的矩陣」，記憶體從隨 context 線性成長變成常數。代價是這個矩陣屬於有損壓縮。

- linear attention 的核心想法
- state update 的數學：decay gate α 與 delta rule β
- 與 Mamba2 / DeltaNet 的關係
- hybrid 3:1 的一次 decode：兩種 cache 同步前進
- chunkwise prefill：長 prompt 怎麼吃
- vLLM 的 Hybrid KV Cache Manager 怎麼同時管兩種 cache

Linear attention 的核心想法

把「查表」換成「維護一個摘要」

full attention 保存每一個位置；linear attention 只保存一個固定大小的摘要矩陣。

FULL ATTENTION

$$o_i = \sum_{j \leq t} \text{softmax}(q_i \cdot k_j) v_j$$

- 必須保留**每一個** k_i 、 $v_i \rightarrow$ KV cache 隨 T 線性成長。
- 讀取量也隨 T 線性成長 $\rightarrow$ 長 context 的 decode 越跑越慢。
- 好處：資訊**無損**。任何位置都能被精確查到。

LINEAR ATTENTION (GDN)

$$S_i = f(S_{i-1}, k_i, v_i) \quad \cdot \quad o_i = S_i q_i$$

- 只保留一個 $d_v \times d_k$ 的矩陣 $S \rightarrow$ 記憶體**固定**。
- 讀取量也固定 $\rightarrow$ decode 速度不隨 context 變慢。
- 代價： S 是**有損壓縮**。寫進去的东西會互相覆蓋。

	FULL ATTENTION	GATED DELTANET
每條序列的記憶體	$O(T)$ 隨 context 線性成長	$O(1)$ 常數
decode 的讀取量	$O(T)$	$O(1)$
prefill 的計算	$O(T^2)$ (可分塊)	$O(T)$ (chunkwise 平行)
資訊保真度	無損	有損，越長越糊
Qwen 的用法	每 4 層放 1 層 (12/48 或 16/64)	其餘 3/4 (36 或 48 層)

混合架構的想法很直接：用 3/4 的層享受 $O(1)$ 的好處，留 1/4 的層做無損查表把細節補回來。現在長 context 模型多半這樣做。

GDN 的一次 state update

decay gate 負責遺忘，delta rule 負責改寫

$S \leftarrow S \cdot \alpha(I - \beta k k^T) + \beta v k^T$ ：先整體衰減、再把 k 這個位址原本存的東西換掉、最後寫入新值。

一個 GDN head 的狀態 S 是固定大小的矩陣 ($d_v \times d_k = 128 \times 128$)



$\alpha_t \in (0,1)$	decay gate：由資料決定要忘掉多少。 $\alpha \rightarrow 0$ 等於整張狀態清空。
$\beta_t \in (0,1)$	delta rule 強度： $\beta \rightarrow 1$ 表示把 k_t 這個「鍵」原本存的東西整條換掉。
S 的大小	$128 \times 128 \times 4 \text{ bytes} = 64 \text{ KiB / head}$ ，和 context 長度完全無關。
代價	S 是有損壓縮。context 越長，被擠掉的細節越多，所以每 4 層要放一層 full attention 來補。

輸出 $o_t = S_t q_t$ ：用 q 去「問」這個摘要矩陣。

Delta rule 的直覺

為什麼不能只是「一直往上加」

寫進去之前先問一次「這個 key 現在存的是什麼」，只修正差額。這樣同一個 key 被更新兩次才不會疊在一起。

為什麼不能只是「一直往上加」？

單純累加（最原始的 linear attention）

$S += \text{apple} \otimes \text{red}$
 $S += \text{apple} \otimes \text{green}$

兩筆疊在一起，查 apple 得到 red 和 green 的混合。

Delta rule：先問，再改

舊值 = $S \cdot k_{\text{apple}} \rightarrow \text{red}$
誤差 = $\text{green} - \text{red}$
 $S += \beta \times \text{誤差} \times k_{\text{apple}}^T$

拆開來看整條式子

$$S_t = S_{\{t-1\}} (\alpha_t (I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T$$

- α_t (decay gate)** 整張 state 乘上一個 0-1 的係數。 $\alpha \rightarrow 0$ 等於一次清空，適合話題切換。
- $I - \beta_t k k^T$ (delta)** 只把 k 這個方向原本存的東西擦掉，其他方向不動。
- $\beta_t v_t k_t^T$** 把新的關聯寫進去。 $\beta \rightarrow 1$ 表示整條換掉。
- 更新的成本** 一次固定維度的矩陣運算 $O(d_k \cdot d_v)$ ，不是「把所有舊 key 跑一遍」，state 裡根本沒有 key 的清單。

Mamba2 只有 α ，DeltaNet 只有 delta。Gated DeltaNet 兩個都有，所以既能快速清空，也能精準改寫。

Mamba2 只有 α ，DeltaNet 只有 delta。[Gated DeltaNet](#) 兩個都有。

三種記憶：Attention、GDN、SSM

同一個問題的三種答案：過去的資訊要怎麼存

檔案櫃、可擦寫的白板、腦中的狀態。差別不在「能不能記」，在「怎麼存」與「存多久不變大」。

Attention

像「檔案櫃」

- 每個 token 一份文件，全部留著
- 查詢時掃過全部
- 原始資訊無損
- 櫃子隨 context 變大

memory $\propto$ T

Gated DeltaNet

像「可擦寫的白板」

- 把 key→value 的關聯壓進一個矩陣
- 可以查、可以改、可以擦
- 白板大小固定
- 寫太多會互相干擾

memory = 常數

SSM / Mamba

像「腦中的狀態」

- 不記逐字逐句，只更新理解
- 新事件進來就更新狀態
- 不是 key→value 字典
- 是動態系統的內部狀態

memory = 常數

同樣一句話塞在 150K token 之前：「Frank 最喜歡的數字是 721938」

Attention

那個 token 的 K/V 還原封不動地在，理論上查得回來。

GDN

被後面 150K 個 token 反覆改寫，可能還在，也可能被蓋掉。

所以 3:1 的混合排列很合理：GDN 提供便宜的壓縮記憶，每 4 層插一層 attention 補回精確查找的能力。
GDN 的 gating（忘掉不重要的）與 delta rule（精準改寫）都是在跟這個有限的容量搏鬥。

所以 3:1 的排列很合理：GDN 提供便宜的壓縮記憶，attention 補回精確查找。

與 Mamba2 / DeltaNet 的關係

同一條家族樹上的三個節點

Mamba2 只會整體遺忘、DeltaNet 只會定點改寫，Gated DeltaNet 把兩種能力合起來。

	更新規則	整體清空記憶	精準改寫某個鍵	VLLM 的歸類
Mamba2	$S_t = \alpha_t S_{t-1} + v_t k_t^T$	快 $(\alpha \rightarrow 0)$	弱	mamba 家族
DeltaNet	$S_t = S_{t-1}(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T$	慢 (一次一個鍵)	強 $(\beta \rightarrow 1)$	—
Gated DeltaNet	$S_t = S_{t-1}(\alpha_t(I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T$	快	強	mamba 家族 (走同一套 state 管理)

為什麼 SERVING 要在意它屬於 MAMBA 家族

- vLLM 把 GDN 當成 mamba 層處理：走同一個 state pool、同一套 `mamba_ssm_dtype` 設定、同一組 shape 計算程式。
- config 裡的 `mamba_ssm_dtype: float32` 就是在說 recurrent 矩陣用 FP32 存，CH6 的 144 MiB 就是這樣來的。
- 所以看 vLLM 的 log 與 metrics 時，GDN 的資源會被標成 mamba。

狀態形狀（照 VLLM 的算法）

$$\text{conv_dim} = d_k \cdot n_k \cdot 2 + d_v \cdot n_v$$

$$\text{state} = (n_v, d_v, d_k)$$

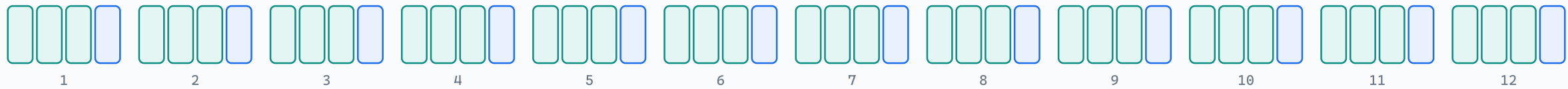
122B：conv_dim 12,288、state (64, 128, 128)
27B：conv_dim 10,240、state (48, 128, 128)

一次 decode 走全部層

兩種 cache 必須同步前進

同一個 token 穿過 48（或 64）層時，GDN 層就地更新固定大小的狀態，attention 層在尾端多寫一格 KV。

Qwen3.5-122B-A10B：48 層 = 12 × (3 × GDN + 1 × full attention)



36 層 GDN：更新 recurrent state

每層 64 個 value head × 128×128 FP32
= 144 MiB / 序列（固定）
讀舊 state → 就地寫回新 state，不會變大

12 層 full attention：append KV

每 token 每層 2 × 2 × 256
= 12 KiB / token（FP8），會一直長大
讀全部歷史 K/V → 尾端多寫一格

兩種 cache 得「同步前進」：任何一邊被換出或算錯位置，整條序列後面的輸出就全壞了。hybrid 模型 serving 最難的地方就在這。

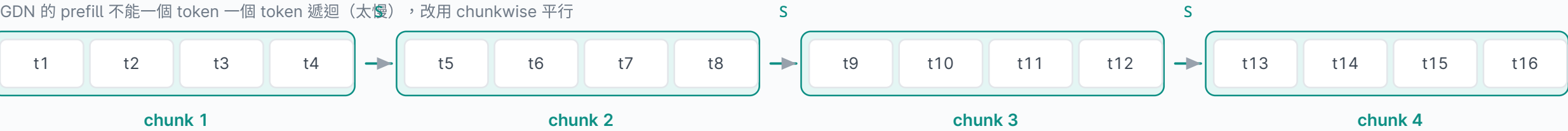
兩者要同步前進。hybrid 模型 serving 最容易出錯的地方就在這裡。

Chunkwise prefill：GDN 怎麼吃長 prompt

chunk 內平行、chunk 之間只傳一個狀態

如果 prefill 也一個 token 一個 token 遞迴，長 prompt 會慢到不能用。GDN 改成分塊：塊內用矩陣乘法，塊間只傳狀態。

GDN 的 prefill 不能一個 token 一個 token 遞迴（太慢），改用 chunkwise 平行



chunk 內	用矩陣乘法平行處理，塞滿 Tensor Core（WY representation）
chunk 之間	只傳遞一個固定大小的狀態 S，序列依賴只剩這一條
full attention 層	同一時間照常做 FlashAttention，並把 prompt 的 K/V 寫進 cache
Prefix caching	vLLM 對 mamba / GDN 的 prefix caching 仍在開發中；混合模型的命中長度取兩種 group 的交集

注意：vLLM 對 mamba / GDN 的 prefix caching 仍在開發中，這會影響多輪對話的 TTFT。

「The gating term only performs elementwise multiplication with intermediate variables without affecting matrix multiply structures.」
Gated Delta Networks: Improving Mamba2 with Delta Rule, arXiv 2412.06464

vLLM 怎麼同時管兩種 cache

Hybrid KV Cache Manager 的三個設計決定

vLLM 把層依 attention 型態分成 KV cache group，強制所有 group 的實體 page 大小一致，再把 mamba state 塞進同樣的 page。

① KV CACHE GROUP

同一個 group 內只能有同一種 attention 型態。混合模型至少兩個 group：full attention 一組、GDN（mamba）一組。分組大小以層數較少的那一種為基準，減少浪費。

② 統一 PAGE SIZE

所有 group 的實體 page 必須同樣大。做法是把 attention 層的 block_size 撐大，直到 $block_size \times kv_hidden \geq mamba_state_size$ ，再把 mamba state 補齊。官方文件坦承這會導致 block_size 超過 400，有效率上的疑慮。

③ PREFIX CACHING 取交集

命中長度是所有 group 的交集：full attention 由左往右掃到第一個 miss；其他型態再從那個長度往回掃。mamba / GDN 的 prefix caching 仍在開發中；超過兩種 attention 型態的模型必須關掉 prefix caching。

這對容量規劃的影響	後果
SSM state 是「每序列一份」，不是「每 token 一份」	併發上限直接乘上 149 MiB（122B）或 153 MiB（27B）。短對話時，這一項會主宰記憶體。
page size 被撐大	小 request 的內部碎片變多；實際可用的 cache 會略少於公式值。
GDN 的 prefix caching 尚未完備	多輪對話的 TTFT 改善幅度會比純 Transformer 小。做 benchmark 時要記錄命中率。
preempt 需要同時處理兩種狀態	換出換回的成本比純 Transformer 高，應盡量避免觸發。

量化：NVFP4 與 FP8

這次要上線的是 nvidia/Qwen3.5-122B-A10B-NVFP4 與 Qwen/Qwen3.8-27B-FP8。量化的影響不只是檔案變小。它決定能不能單卡跑，也決定哪些張量還留在 **BF16** 當頻寬大戶。

- NVFP4 與 FP8 的位元佈局
- checkpoint 實際量化了哪些張量（直接看 weight map）
- 省下多少：權重、**KV cache**、可用 cache 空間
- 精度代價：NVIDIA 官方的評測表
- 為什麼量化不會移動**屋頂線**的轉折點

NVFP4 與 FP8 的位元佈局

兩層 scale 是 NVFP4 能在 4 bit 保住精度的關鍵

NVFP4 = 每 16 個 4-bit 值共用一個 **FP8** scale，再乘一個 per-tensor 的 FP32 scale，平均 4.5 bit／值。

一個 NVFP4 block：16 個 4-bit 值 + 1 個 FP8 block scale



16 個 E2M1 (1 符號 + 2 指數 + 1 尾數 = 4 bit)
可表示的非零大小只有 {0.5, 1, 1.5, 2, 3, 4, 6}

block scale
FP8 E4M3 (8 bit) ／每 16 個值一份

tensor scale
FP32／每個張量一份 (weight_scale_2)

→ 平均 $(16 \times 4 + 8) / 16 = 4.5 \text{ bit / 值}$

對照：Qwen3.8-27B-FP8 用的 blockwise-128

128 個 FP8 E4M3 值 (每個 8 bit)

1 個 FP32 scale

→ 8.03 bit / 值

- 為什麼 block 要小** block 越小，一組值裡的動態範圍越窄，4 bit 的 8 個大小級距就越夠用。NVFP4 用 16，MXFP4 用 32。
- 為什麼 scale 要用 FP8 而不是 2 的冪次** E4M3 是真正的浮點數，能表示 $1.75 \times$ 這種比例；MXFP4 的 E8M0 只能表示 2^n ，誤差較大。代價是 scale 的儲存開銷加倍。
- 兩層 scale 的分工** block scale 處理局部差異，tensor scale 處理整體量級。兩者相乘才還原成原始權重。

對照 FP8：Qwen3.8-27B-FP8 用 128 個值共用一個 scale，平均 8.03 bit / 值。

checkpoint 實際量化了哪些張量

直接看 safetensors 的 weight map，不用猜

122B 的 NVFP4 只量化了 routed expert；GDN、attention、shared expert、lm_head、MTP head 全部還是 BF16。

從 safetensors 的 weight map 直接看出來：誰有 weight_scale，誰就被量化了



被量化成 NVFP4	48 層 × 256 個 routed expert 的 gate/up/down = 116.0 B 參數
維持 BF16 不量化	全部 36 層 GDN、12 層 full attention、48 個 shared expert、router、embedding、lm_head、視覺塔、MTP head



被量化成 FP8	64 層 dense FFN、16 層 attention 的 q/k/v/o、48 層 GDN 的 in_proj_qkv / in_proj_z / out_proj、MTP head
維持 BF16 不量化	GDN 的 in_proj_a / in_proj_b / conv1d / A_log / dt_bias、所有 norm、embedding、lm_head、視覺塔

驗算：48 × 256 × 3 × 3072 × 1024 = 115,964,116,992 個 4-bit 權重 → 57,982,058,496 bytes，與 checkpoint 的 U8 統計完全一致；
÷ 16 = 7,247,757,312 個 FP8 block scale，也與 F8_E4M3 統計完全一致。27B 的 FP8 參數量同樣可以逐張量湊到分毫不差。

怎麼確定的：只有帶 weight_scale 的張量才是被量化的。把數量乘出來，與 HuggingFace 回報的 dtype 統計逐一比對。

省了多少

權重、KV cache 與可用 cache 空間的三筆帳

122B 從 250.17 GB 降到 83.47 GB。這正好是「單卡跑得動」與「至少要兩張卡」的分界線。

	BF16 權重	量化後權重	省下	單張 B200 180 GB	剩給 CACHE (UTIL 0.90，扣 8 GiB ACTIVATION)
Qwen3.5-122B-A10B NVFP4	250.17 GB	83.47 GB	67%	單卡放得下 (BF16 需 2 卡)	65.1 GiB
Qwen3.8-27B FP8	55.56 GB	30.87 GB	44%	單卡輕鬆	114.1 GiB
Qwen3-32B BF16	65.52 GB	—	—	單卡放得下	81.9 GiB

KV CACHE 也可以量化

NVIDIA 的 NVFP4 模型卡在部署指令裡 直接帶了 `--kv-cache-dtype fp8`，把 KV 從 2 bytes 降到 1 byte。
122B：24 → 12 KiB / token，等於同樣的 HBM 能放兩倍的 context。

SSM STATE 不吃量化的紅利

GDN 的 recurrent state 由 `mamba_ssm_dtype: float32` 決定，是 FP32，與權重量化無關。
兩個模型都是 144 MiB / 序列。想省的話要另外設 `--mamba-ssm-cache-dtype`，但要驗證品質。

被忽略的一塊：MTP HEAD

122B 的 MTP head 有 2.52 B 參數而且沒有被量化，BF16 佔 5.05 GB。
只要開 speculative decoding，這 5.05 GB 就從 cache 預算裡扣掉，相當於少放 20 條 8k 序列。

精度代價

NVIDIA 官方的評測：NVFP4 對 FP8 的差距在 0.6 分以內

五項評測裡四項小輸、一項小贏，最大差距 0.60 分。以 4× 的記憶體節省來說，這個代價很划算。

BENCHMARK	FP8 BASELINE	NVFP4	差距
MMMU Pro	75.90	75.55	-0.35
GPQA Diamond	87.37	86.77	-0.60
SciCode	42.16	41.79	-0.37
AA-LCR	65.50	67.13	+1.63
IFBench	70.91	70.80	-0.11

數據取自 nvidia/Qwen3.5-122B-A10B-NVFP4 模型卡，對照組是同一家的 FP8 版本（不是 **BF16**）。查核日 2026-09-08。

要注意的三件事

- 對照組是 **FP8** 不是 BF16。FP8 對 BF16 本身也有一點差距，兩段要分開看。
- 這些是**單輪**評測。多輪 agent 任務的誤差會累積，官方沒有提供這類數據。
- 量化對**長尾行為**（罕見格式、少數語言、極長 context）的影響通常比平均分數大。自家的驗收集一定要自己跑一次。

27B 的 FP8

Qwen 官方模型卡對 FP8 版本的說法是「performance metrics nearly identical to those of the original model」，沒有附評測表。
FP8 blockwise-128 在業界已算成熟做法，但同樣建議自己跑一次驗收集。

「Weights and activations of the linear operators within transformer blocks in **MoE** are quantized.」

nvidia/Qwen3.5-122B-A10B-NVFP4 模型卡

量化不會移動屋頂線的轉折點

它讓兩邊同時變快，你的相對位置沒動

BF16、**FP8**、**NVFP4** 的 N^* 都是 292。量化把讀取和運算**同時**加速，比值不變。

三種精度、同一個轉折點

$$N^* = F \cdot b / (2 \cdot BW)$$

精度	B (B/參數)	F (PFLOP/S)	N^* TOKEN/STEP
BF16	2	2.25	292
FP8	1	4.5	292
NVFP4	0.5	9.0	292

Blackwell 每把位寬砍半就把張量核心吞吐加倍。分子的 F 加倍、b 減半，剛好相消。

那量化到底改善了什麼

- **絕對速度**：同一個 batch 下，step 時間變短。122B 從 BF16 的 250.17 GB 降到 83.47 GB，單就權重讀取就快 3.0 倍。
- **裝得下**：122B 從必須兩卡變成單卡，省掉 TP 的 all-reduce。
- **cache 空間**：省下來的 **HBM** 全部變成併發數。
- **沒有改善**：你還是 memory-bound，還是需要靠 batch 或 **speculative decoding** 才能把算力用起來。

所以量化與 speculative decoding 是**互補**而不是替代：量化讓每一步更便宜，speculative decoding 讓每一步做更多事。兩個都做才會同時改善 **TPOT** 與 throughput。

單卡記憶體帳本

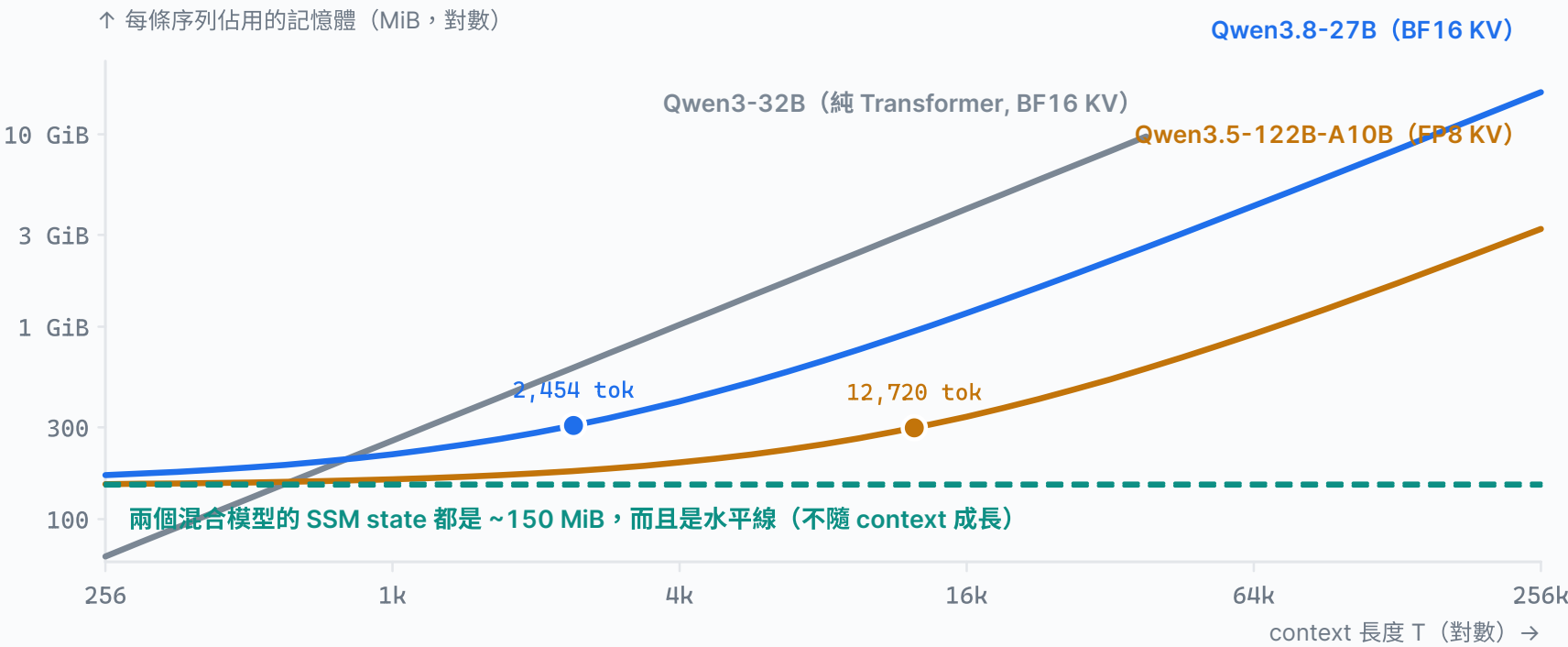
180 GB 的 HBM 要同時放權重、起草器、KV cache、SSM state 與 activation。混合模型的關鍵在於：**KV cache 與 SSM state 必須分開算**，它們的成長曲線完全不同。

- 兩種 cache、兩種成長曲線
- KV cache 的公式與逐模型數字
- SSM cache 的公式與逐模型數字（含 speculative decoding 的影響）
- 交會點：context 多長時 KV 才追上 SSM
- 互動試算：B200 單卡的完整帳本

兩種 cache、兩種成長曲線

一條斜線，一條水平線

KV cache 隨 context 線性成長；**SSM state** 是常數。混合模型在短 context 時，記憶體其實由 SSM 主導。



交會點的意義

context 短的時候，混合模型的記憶體幾乎全部是固定的 SSM state；長 context 之後才換成 KV cache 主導。

122B : 12,720 token

27B : 2,454 token

→ 短對話的併發上限其實卡在 SSM state，不在 KV。

兩條線的交會點就是「KV 開始接管」的位置。

KV cache 的容量計算

先算每條序列，再考慮並行、分片與配置開銷

只有 full attention 層需要 KV。混合架構把層數從 64 降到 12-16，直接把 KV 砍掉 3/4。

$$M_{KV} = 2 \times L_{full} \times H_{KV} \times d_{head} \times bytes \times T \quad (2 = K \text{ 與 } V \text{ 各一份})$$

模型	L_FULL	H_KV	D_HEAD	KV/TOKEN <u>BF16</u>	KV/TOKEN <u>FP8</u>	@32K (FP8)	@262K (FP8)
Qwen3.5-122B-A10B <u>NVFP4</u>	12	2	256	24 KiB	12 KiB	384 MiB	3.00 GiB
Qwen3.8-27B <u>FP8</u>	16	4	256	64 KiB	32 KiB	1,024 MiB	8.00 GiB
Qwen3-32B <u>BF16</u>	64	8	128	256 KiB	128 KiB	4.00 GiB	超過原生 context

粗體的是本次的預設

122B 依 NVIDIA 模型卡的部署指令用 `--kv-cache-dtype fp8`；27B 沒有這個建議，預設 auto（BF16）。試算器可以切換兩者比較。

公式之外還要加什麼

- PagedAttention 的最後一塊尾巴（block 內部碎片）
- 混合模型撐大的 page size 造成的額外碎片
- preempt 時的搬移緩衝

實務上抓公式值的 1.05-1.15 倍。

多卡時怎麼變

Tensor parallel 會把 KV head 切開：TP=2 時每張卡的 KV/token 減半。但 122B 只有 2 組 KV head，**TP 最多只能切到 2**，再多就得複製。這是 GQA 的副作用。

SSM cache 的容量計算

固定大小的 recurrent 矩陣，加一小段 convolution 歷史

每條序列固定 144 MiB 的 FP32 矩陣，與 context 長度完全無關。conv state 很小，但會隨 speculative decoding 的 k 變長。

$$M_{state} = L_{GDN} \times H_V \times d_V \times d_K \times 4 \text{ B} \quad M_{conv} = L_{GDN} \times conv_dim \times (conv_kernel - 1 + k_{spec}) \times 2 \text{ B}$$

模型	L_GDN	H_V	D_V × D_K	RECURRENT (FP32)	CONV 無 SPEC	CONV 有 SPEC	合計 / 序列
Qwen3.5-122B-A10B	36	64	128 × 128	144.0 MiB	2.53 MiB	5.06 MiB k=3	149.1 MiB
Qwen3.8-27B	48	48	128 × 128	144.0 MiB	2.81 MiB	9.38 MiB k=7	153.4 MiB

144 MiB 的巧合

兩個模型的 **recurrent state** 竟然一樣大：122B 是 36×64 = 2,304，27B 是 48×48 = 2,304 個 value head，每個都是 128×128×4 B = 64 KiB。

好記，但也提醒：**SSM state 與模型大小無關**，只跟 GDN 層數 × head 數有關。

為什麼是 FP32

config 的 `mamba_ssm_dtype: "float32"`。recurrent 狀態會被反覆乘加上千次，用 **BF16** 累積會漂移。

vLLM 有 `--mamba-ssm-cache-dtype` 可以改成 BF16，直接砍半成 72 MiB，但長 context 的品質要先驗過。

SPEC DECODING 的影響

vLLM 的 `conv_state_shape = (conv_dim, conv_kernel - 1 + num_spec)`。

27B 開 **DFlash2** (k=7) 之後，conv 長度從 3 變成 10，conv state 從 2.81 漲到 9.38 MiB。佔比不大，但「開 spec 會多吃記憶體」有一部分就是它。

交會點：context 多長時 KV 才追上 SSM

短對話的併發上限其實由 SSM state 決定

122B 要到 12,720 token、27B 要到 2,454 token，KV cache 才追上固定的 SSM state。

12,720tok
122B (FP8 KV) 的交會點

2,454tok
27B (BF16 KV) 的交會點

4,908tok
27B 若改用 FP8 KV

0tok
Qwen3-32B：沒有 SSM state，從第一個 token 就是 KV

情境	典型 CONTEXT	122B 每條序列	27B 每條序列	誰主導
短問答 / 分類	1k	161 MiB	217 MiB	SSM state 佔 90%+
一般對話	8k	245 MiB	665 MiB	SSM 仍佔多數 (122B)
RAG / 長文件	32k	533 MiB	2,201 MiB	KV 開始主導
Agent 長對話	128k	1,685 MiB	8,345 MiB	KV 完全主導
跑滿原生 context	262k	3,221 MiB	16,537 MiB	KV 完全主導

實務結論：如果你的 workload 是短對話，把 `--max-model-len` 調小**不會**提高多少併發，因為瓶頸在 SSM state；這時要調的是 `--max-num-seqs` 或考慮把 SSM state 降成 BF16。反之如果是長文件場景，KV 的精度與 context 上限才是主要旋鈕。

互動試算：B200 單卡的完整帳本

自己調參數看併發上限怎麼變

權重 + 起草器 + KV + SSM + activation 必須全部塞進 gpu_memory_utilization 圈起來的空間。



activation 估計值僅為教學用途；實際由 vLLM 依 max_num_batched_tokens、CUDA graph 捕捉的 batch 尺寸與 kernel workspace 決定。權重採用 HuggingFace 上實際 safetensors 位元組數。

調哪個旋鈕影響最大

旋鈕	效果
<code>--kv-cache-dtype</code> fp8	KV 直接砍半；長 context 場景收益最大
<code>--max-num-seqs</code>	同時乘上 KV 與 SSM，短對話場景的主要旗標
關掉 <code>speculative decoding</code>	省下起草器權重與加長的 conv state

這個試算沒有算進去的

- `PagedAttention` 最後一塊 block 的內部碎片
- 混合模型撐大 page size 造成的碎片
- CUDA context 與 allocator 保留區

實務上把公式值乘 1.05–1.15 比較保險。

這是記憶體上限，不等於「能達成 SLO 的併發數」。能承諾的容量由 CH9 的開環測試決定，那個數字通常小得多。

併發上限與 context 的取捨

同一張卡，你可以選「多人短對話」或「少人長文件」

HBM 是固定的：併發數 × 每條序列的記憶體 ≤ cache 預算。這是一條雙曲線，沒有免費的午餐。

KV 精度

FP8

BF16

context 長度

8,192 tok



■ KV cache (隨 context 線性成長) ■ SSM + conv state (固定)

模型	KV / TOKEN	KV @ 8,192	SSM STATE	合計 / 序列	SSM = 幾個 TOKEN 的 KV
Qwen3-32B	128 KiB	1,024 MiB	—	1,024 MiB	—
Qwen3.8-27B	32 KiB	256 MiB	153.4 MiB	409.4 MiB	4,908
Qwen3.5-122B-A10B	12 KiB	96 MiB	149.1 MiB	245.1 MiB	12,720

122B @ 單卡 B200

cache 預算約 60 GiB (已扣起草器)

4k	314 條
32k	116 條
262k	19 條

27B @ 單卡 B200

cache 預算約 111 GiB (已扣起草器)

4k	277 條
32k	51 條
262k	7 條

這些是**記憶體**能容納的序列數上限，不是「能達成 **SLO** 的併發數」。能承諾的容量還受延遲限制，CH9 的開環測試會找出那個更小的數字。記憶體上限通常遠大於 SLO 上限。

同一顆模型，兩個引擎怎麼記帳

vLLM 一個統一池，SGLang 兩個獨立池

兩邊 log 印的「KV cache」不是同一個東西：vLLM 把 GDN state 混進同一個 pool 用 token 計價，SGLang 把 KV 與 SSM 分成兩個池分開配。數字不能直接相比。

VLLM：統一 PAGE，一個數字

把 attention 的 `block_size` 撐大到 $\text{page} \geq \text{mamba state}$ ，兩種 cache 用同樣大小的 page 共享一個 block pool。所以 log 只印一行 `GPU KV cache size: N tokens`——**GDN state 已經混在裡面**。

block size	832 tokens
mamba page padding	1.71%
池子 (TP=2 合計)	283 GiB
印出來的容量	3,339,380 tokens
262K 併發	12.74×

用本章的 SSM 公式反算 mamba page，只有 block size 832 能對上 1.71% 這個 padding——等於替公式做了一次實機驗算。

SGLANG：兩個池，分開印

`token_to_kv_pool` 按 **token** 配，mamba/SSM pool 按 **槽位** 配（每槽 = 一條序列的完整 state，與 context 無關）。所以會看到兩個獨立的數字。

KV 池	31 GiB (508,278 tokens)
SSM 池	97 GiB
換算槽位	≈ 677 條
262K 併發 (KV 限)	508,278 ÷ 262,144 = 1.94

同一顆 27B、單卡、沒開 spec。兩個池**不能互相調度**，所以上限是兩者取小。

SGLang 這組的配比是歪的。SSM 池預留了 677 條序列的空間，但 KV 只剩 31 GiB——跑滿 262K 時只放得下 **1.9 條**。把槽位砍到接近實際併發（例如 64 條，約 9.2 GiB），省下的還給 KV，同一張卡的 262K 併發可以拉到約 7.4 條。vLLM 的統一池則是自動按需分配：不用調，但也看不出哪一塊吃掉了記憶體。

Speculative decoding 對 serving 的影響

本次上線的設定：**Qwen3.5-122B-A10B** 用內建 MTP 猜 3 個、**Qwen3.8-27B** 用 DFlash2 猜 7 個。這一章把它從「一個開關」拆成 機制、接受率、記憶體、批次上限與 SLO 五個面向。

- 為什麼可以猜：decode 有閒置算力
- draft → verify → accept / reject 的一輪
- MTP 與 DFlash2 的機制差異與時間軸
- 接受長度 τ ：報酬遞減得多快
- $(k+1) \times$ 的 token 撞上屋頂線 → 併發上限
- 長 context 會吃掉接受率（實測案例）
- 記憶體成本、與 prefix caching 的衝突、什麼時候該關掉

為什麼可以「猜」

decode 時算力大量閒置，那塊算力是免費的

32 併發時，27B 只用掉不到 15% 的張量核心算力。speculative decoding 就是把那 85% 拿去猜 token。

decode 時，GPU 的兩種資源使用率差距很大



speculative decoding 就是打這塊閒置算力的主意：拿它去猜後面幾個 token，再讓目標模型用同一次權重讀取一口氣驗證。猜對就一次前進好幾格。

輸出品質不變
驗證用 rejection sampling：逐位置接受或拒絕，數學上與不猜時的輸出分布相同。

但這塊算力不是無限的
併發一高，閒置算力就被用光；此時再猜就是純浪費。本章要量的就是那條界線。

輸出品質不變（rejection sampling 保證），但那塊算力有限，本章要量的就是那條界線。

一輪 draft → verify → accept

被拒絕之後整串丟掉，但一定會前進至少一格

起草 k 個 → 目標模型一次驗證 k+1 個位置 → 從左到右接受，遇到第一個拒絕就停，並補上一個保底 token。

而

模型

推論

的

瓶頸

起草 k=7

一次產生 7 個草稿

驗證

目標模型一次 forward 同時驗證全部 8 個位置（1 個已知 + 7 個草稿）

結果

第 4 個起被拒絕
→ 後面整串丟掉

↑ 保底 token

這一輪前進了 4 格（3 個被接受的草稿 + 1 個保底 token），只花了 1 次起草 + 1 次目標模型 forward。

被接受的 token	確定與不猜時完全一樣（rejection sampling 保證）
被拒絕之後	整串後面都丟掉，因為它們是以錯誤的前綴為條件產生的
保底 token	拒絕發生的位置，直接採用目標模型自己的輸出。所以最差情況也會前進 1 格，就 token 數而言不會比不猜慢

這一輪前進 4 格，只花了 1 次起草 + 1 次目標模型 forward。

MTP 與 DFlash2：兩種完全不同的起草器

一個是序列式跑 k 次，一個是一次猜完整塊

MTP 的起草成本隨 k 線性成長；DFlash2 的 block diffusion 一次 forward 出整塊，成本與 k 無關。

同樣要產生 8 個位置的草稿，兩種起草器的時間軸完全不同



差別在成本結構：MTP 的起草成本隨 k 線性成長 ($T_{\text{draft}} = k \times t_{\text{step}}$)，DFlash2 的起草成本與 k 無關 ($T_{\text{draft}} = t_{\text{parallel}}$)。

MTP (Qwen3.5 內建)
1 層 decoder，autoregressive 跑 k 次。與目標模型同時訓練，接受率高；但每一次都要再過 lm_head，k 一大就不划算。

DFlash2 (block diffusion)
獨立的 2 B 起草模型，用非因果 attention 同時看目標模型的 hidden state 與整排 mask token，一次 forward 產生整塊；再用輕量 selector 挑一條路徑。

DFlash2 敢用 k = 7、MTP 通常停在 k = 2-4，差別就在這裡。

兩個起草器的規格對照

本次上線設定的完整參數

122B：內建 MTP、k=3。27B：外掛 DFlash2、block size 8、k=7。兩者的成本結構完全不同。

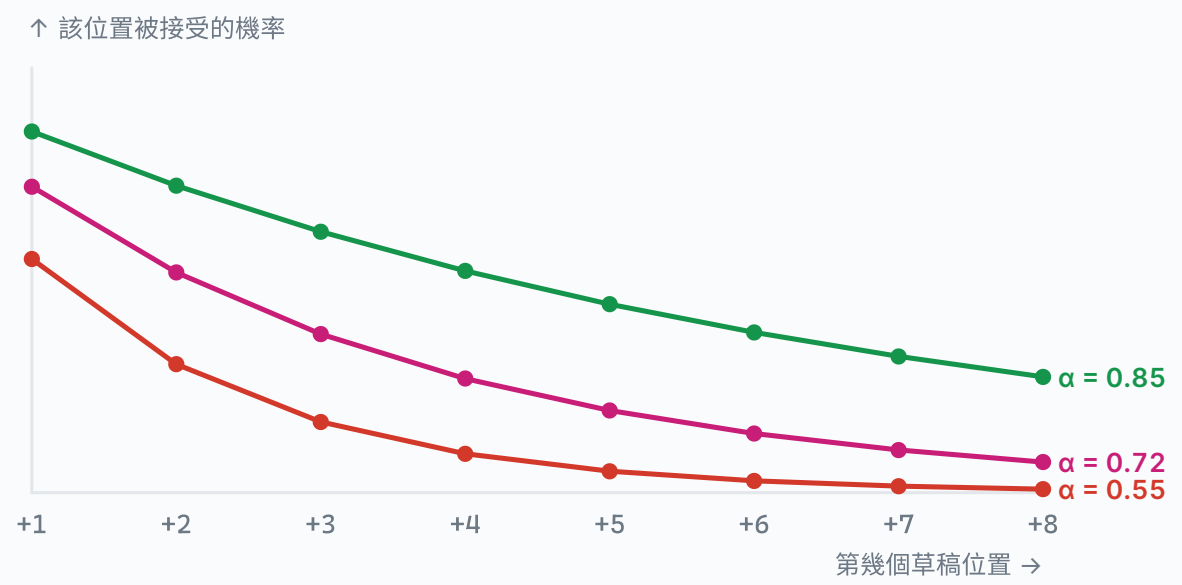
	QWEN3.5-122B-A10B	QWEN3.8-27B
起草方法	MTP （multi-token prediction）， <u>EAGLE</u> 系	DFlash2 ， <u>block diffusion</u>
來源	模型 checkpoint 內含 <code>mtp.*</code> （Qwen 官方訓練）	外部 checkpoint <code>incoai/Qwen3.8-27B-DFlash2</code>
起草器規模	1 層 <u>MoE</u> decoder，2.52 B 參數（ <u>BF16</u> ，未量化）	5 層 sliding attention，1.92 B 參數（BF16）
草稿 token 數 k	3 （官方建議 2-4）	7 （block size 8，扣掉 anchor）
起草方式	autoregressive，跑 k 次 forward	一次 forward 產生整塊
起草成本	$\propto k$ （每次都要重讀 lm_head，1.53 GB）	固定 3.85 GB，與 k 無關
條件輸入	目標模型最後一層的 hidden state	目標模型 5 層 hidden state 注入到起草器每層的 KV
vLLM 設定	<code>{"method": "qwen3_next_mtp", "num_speculative_tokens": 3}</code>	<code>{"method": "dflash", "model": "incoai/Qwen3.8-27B-DFlash2", "num_speculative_tokens": 7}</code>
官方實測接受長度	官方未公布逐 benchmark 數字（本報告取 $\tau \approx 2.6$ 作保守估計）	GSM8K 5.46 、MATH-500 5.28 、MBPP 4.79 、HumanEval 4.39
官方實測加速（concurrency 1）	—	GSM8K 3.43× 、MATH-500 3.34× 、HumanEval 3.11×

DFlash2 模型卡的吞吐數字是在 concurrency 1 量的，且未載明硬體。**接受長度**可以跨硬體沿用（它只跟模型與資料有關），**絕對 tok/s 與加速倍數不行**，那會隨 GPU、量化與併發改變。上線前要自己量一次。

接受長度 τ ：報酬遞減得很快

k 加大不會等比例變好

每個草稿位置能被接受的機率是 α^i ，所以 $\tau = (1 - \alpha^{(k+1)}) / (1 - \alpha)$ ，很快就飽和。



期望接受長度 $\tau = \sum \alpha^i = (1 - \alpha^{(k+1)}) / (1 - \alpha)$

α	k=1	k=3	k=7	k=15
0.55	1.55	2.02	2.20	2.22
0.72	1.72	2.61	3.31	3.55
0.80	1.80	2.95	4.16	4.86
0.85	1.85	3.19	4.85	6.17
0.90	1.90	3.44	5.70	8.15

報酬遞減很快
 $\alpha = 0.80$ 時，k 從 7 加到 15， τ 只從 4.16 長到 4.80 (+15%)，但驗證成本翻倍。

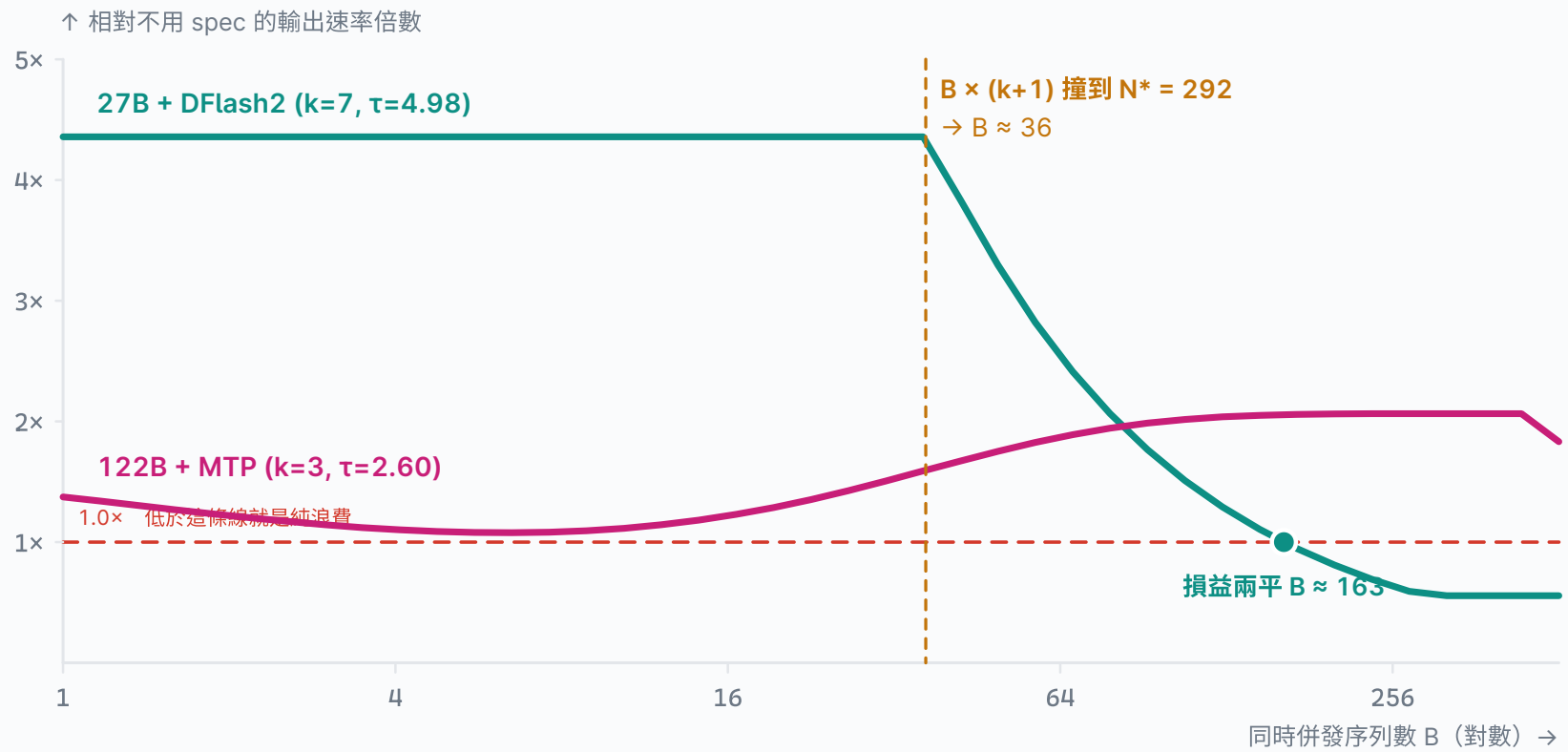
實測參考：DFlash2 在 Qwen3.8-27B 上，concurrency 1 的接受長度是 GSM8K 5.46、MATH-500 5.28、MBPP 4.79、HumanEval 4.39；對應 k = 7、 $\alpha \approx 0.80-0.86$ 。EAGLE-3 論文在 LLaMA-3.1-8B 上報告 $\tau = 6.23$ 。

實測參考：DFlash2 在 27B 上的接受長度是 4.4–5.5，對應 $\alpha \approx 0.80-0.86$ 。

(k+1)× 的 token 撞上屋頂線

這是 speculative decoding 對 serving 最大的影響

驗證階段一次要送 $B \times (k+1)$ 個 token。一旦超過 $N^* \approx 292$ ，就從「免費」變成「多花錢」。



為什麼兩條線形狀不同

27B 是 dense 模型：每個 step 讀的權重固定， $B \times 8$ 一旦超過 N^* 就變成 compute-bound，加速直線墜落。

122B 是 MoE：batch 越大，被碰到的 expert 越多、要讀的權重也越多，所以低併發時 spec 的優勢反而被吃掉；但高併發時 expert 讀取被更多 token 分攤，優勢回升。

紅色虛線是損益兩平線。低於它，speculative decoding 就是純粹在燒算力。

一次驗證 8 個位置：為什麼可行、什麼時候變貴

驗證是一次迷你 prefill，不是 8 次 decode

草稿已經在手上，所以目標模型可以用一般的 causal mask 一次算完 8 個位置的分布——共用同一次權重讀取。代價是 $B \times 8$ 這個數字會去撞 N^* 。

為什麼 AUTOREGRESSIVE 還能一次驗證 8 個

驗證階段目標模型**不需要自己生成**，它只回答一個問題：「如果前綴長這樣，我會吐什麼？」把 anchor 接上 7 個草稿拼成一條序列送進去，配 causal mask 一次 forward，第 i 個位置看到的正好是它前面那些 token，跟逐步生成看到的**完全一樣**。

所以它是一次**迷你 prefill**。8 個位置的 GEMM 一批做完，權重只讀一次——這就是 N 從 B 變成 $B \times (k+1)$ 的由來。平均只有 $\tau \approx 5.0$ 個位置會被採用，其餘算完就丟。

27B 的梯子 (K = 7)

B	$B \times 8$	VS N^*	位置
8	64	0.22×	免費區
16	128	0.44×	免費區
32	256	0.88×	免費區
36	288	0.99×	碰線
48	384	1.31×	開始收費
64	512	1.75×	開始收費
128	1,024	3.50×	開始收費

兩個門檻不要搞混。 $B \approx 36$ 是「額外驗證不再免費」的**起點**，不是「超過就不能開」。真正由賺轉賠是損益兩平點，解析模型算出來是 **$B \approx 163$** ： $B=32$ 還有 $4.36\times$ 、 $B=64$ 剩 $2.54\times$ 、 $B=128$ 只剩 $1.27\times$ 。實機 $\text{max_num_seqs} = 60$ 正好落在這段邊際效益快速遞減的區間，值得逐點 A/B。

互動試算：spec decoding 划不划算

自己調 k、α 與併發數

三個變數決定一切：草稿數 k、逐位置接受率 α、同時併發數 B。

模型 / 起草器

27B + DFlash2122B + MTP

草稿 token 數 k 7

逐位置接受率 α 0.80

同時併發序列 B 16

4.16tok

期望接受長度 $\tau = (1-\alpha^{k+1})/(1-\alpha)$ ，上限 8

3.64x

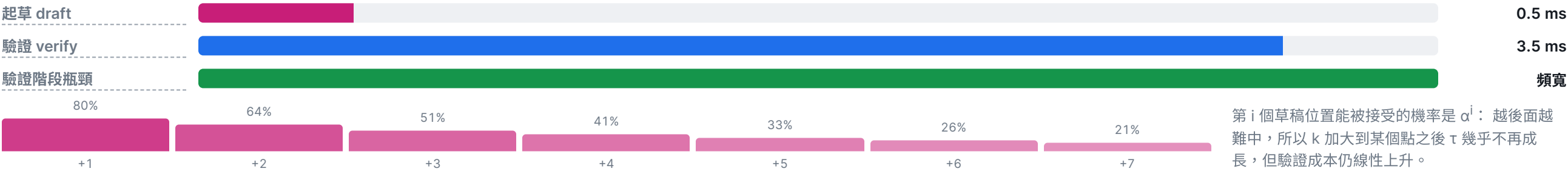
相對不用 spec 的輸出速率

0.96ms

TPOT：3.5 ms → 0.96 ms

136

損益兩平的併發數（超過就別開）



建議的操作順序

1. 先用預設值看 27B + DFlash2 在低併發的 4.4x
2. 把 B 拉到 64 → 加速掉一半
3. 再拉到 200 → TPOT 比不開還差
4. 把 k 降到 3 → 高併發時反而更好
5. 把 α 降到 0.6（模擬長 context）→ 整條塌下來

模型的假設

• 只算權重讀取與張量運算（roofline 上限）
• MoE 的 routing 假設均勻 → 專家碰觸數是上界
• 每個草稿位置的接受率固定為 α（實際會逐位置下降）
• 不含 attention kernel、all-to-all、取樣與排程開銷

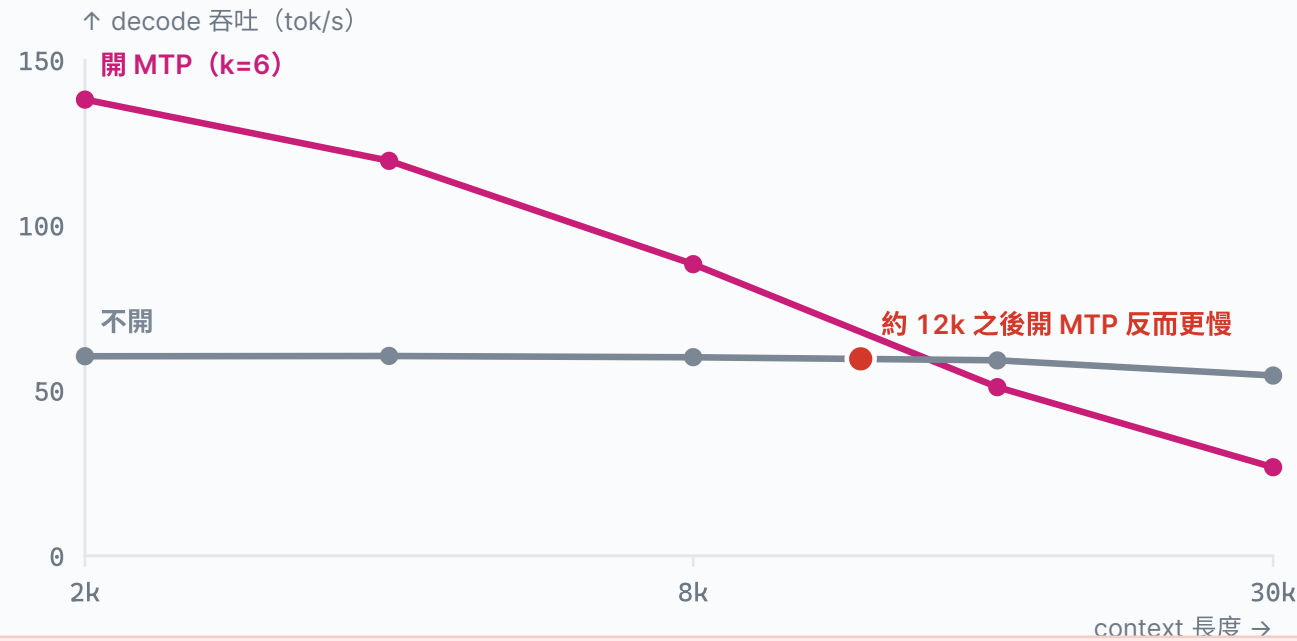
所以怎麼用它

當成形狀的指南，不是絕對值的預測。它能回答「B 大概到哪裡就該關掉」，不能回答「會有幾 tok/s」。量過一次實測之後，把「實測 ÷ 解析上限」的比例記下來，之後就能用它快速估算新設定。

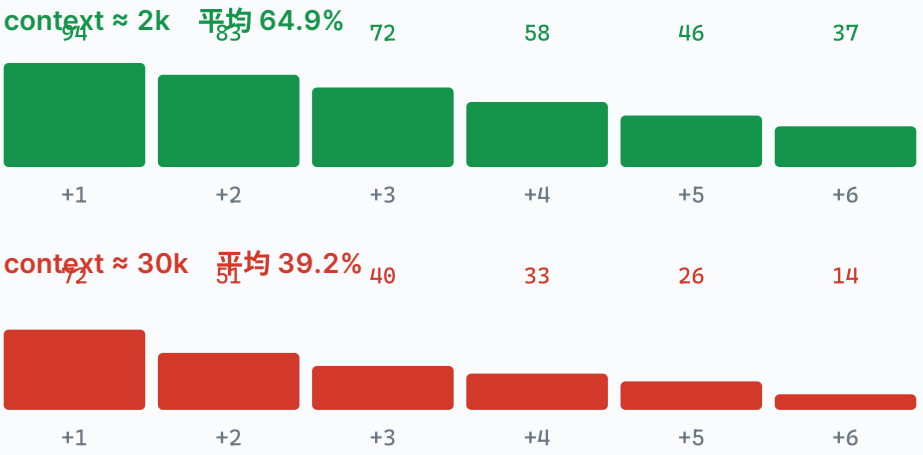
長 context 會吃掉接受率

實測案例：從加速 129% 變成減速 51%

接受率不是常數。同一個模型、同一個 k，context 從 2k 長到 30k，平均接受率從 65% 掉到 39%。



逐位置接受率也整條下降



資料來源：vLLM issue #47602 在 Qwen3.6-27B 上的實測。context 從 2k 長到 30k，平均接受率從 64.9% 掉到 39.1%，吞吐從「比不開快 129%」變成「比不開慢 51%」。長 context 服務一定要重新量接受率，不能沿用短 prompt 的結論。

長 context 服務要重新量接受率，短 prompt 的結論搬不過來。

其他三個副作用

記憶體、prefix caching 與延遲抖動

開 spec 不只是多一個開關：它會吃記憶體、干擾 prefix caching，也會讓 ITL 變得不規則。

開了 speculative decoding，記憶體帳本會多出三筆

① 起草器的權重

一次性成本，直接從 cache 預算裡扣掉

122B · MTP head

5.05 GB

27B · DFlash2

3.85 GB

② GDN 的 conv state 變長

vLLM 的 conv_state_shape = (conv_dim, conv_kernel-1+num_spec)，每條序列都要多留

122B · 4-1+3 = 6

2.53 MiB

27B · 4-1+7 = 10

6.56 MiB

③ 每條序列要多預留 k 個 KV slot

驗證時 k+1 個位置的 K/V 都要有地方放；被拒絕的再釋放

122B · k=3

36.00 KiB

27B · k=7

0.44 MiB

算總帳（單卡 B200、gpu_memory_utilization 0.90）：

122B：起草器 5.05 GB 是固定支出，相當於少放 20 條 8k 序列。

27B：起草器 3.85 GB，每條序列的 state 也從 146.8 MiB 漲到 153.4 MiB（+4.5%）。

記憶體不是 spec decoding 的主要限制。但 cache 本來就很緊的長 context 場景，這幾 GB 會直接吃掉併發數。

總結：記憶體不是主要限制，但在 cache 已經很緊的長 context 場景，這幾 GB 會直接吃掉併發數。

干擾 PREFIX CACHING

vLLM issue #38182 回報：在 Qwen3.5-35B-A3B 上開 MTP（`num_speculative_tokens: 1`）之後，prefix cache 命中率從 **92% 掉到 71%**。

原因是被拒絕的草稿會讓序列長度不對齊 block 邊界，使得可快取的 block 變少。對 agent／多輪對話（本來就靠 prefix caching 吃飯）影響特別大。

延遲變得不規則

沒開 spec 時，每個 step 產生 1 個 token，ITL 很平穩。開了之後，一輪可能吐 1 個也可能吐 8 個。平均 TPOT 改善，但 ITL 的 **p99** 會變差。如果 SLO 是用 ITL p99 定義的，要重新校準門檻，或改用 TPOT。

什麼時候該開、什麼時候該關

一張可以直接用的決策表

低併發 + 短 context + 高接受率 → 開，而且 k 可以大。高併發或長 context → 調小 k 或關掉。

情境	建議	理由
單一使用者互動、demo、內部工具 (B ≤ 8)	全開，k 用最大	整段都在 memory-bound 區，B×(k+1) 遠低於 292。TPOT 可以改善 2-4×。
中等併發 (B ≈ 8-36)、context < 8k	開	仍在 memory-bound 區。27B 配 k=7 可以撐到 B ≈ 36。
高併發 (B > 36)、追求整機 throughput	調小 k，或關掉	驗證階段變成 compute-bound，多猜的 token 都在燒算力。
長 context (> 16k)	先量再決定	接受率會顯著下降，實測案例顯示 30k 時反而慢 51%。
前綴重複度高的 agent／多輪對話	A/B 測 prefix cache 命中率	spec 會拉低命中率，兩個效益可能互相抵消。
混合負載、QPS 波動大	用動態 speculative decoding	vLLM 支援依負載調整；SmartSpec 的做法是依 goodput 決定每輪的 k，高負載自動歸零。

上線前一定要量的三個數字

- 逐位置接受率與 τ (引擎的 acceptance metrics)
- 開／關 spec 的 goodput 對照 (同一組 SLO)
- prefix cache 命中率的變化

上線後要盯的訊號

- τ 隨時間下降 → workload 變了，該重評 k
- ITL p99 惡化但 TPOT 改善 → SLO 定義要調整
- preempt 次數上升 → 起草器吃掉的記憶體造成壓力

本次設定的初步判斷

122B + MTP k=3：建議開啟，MoE 的特性讓它在高併發時仍有優勢。

27B + DFlash2 k=7：建議在 B ≤ 36 時開啟；高併發批次任務可考慮降到 k=3 或關閉。兩者都要先用自家 workload 量接受率。

上線設定與命令

可以直接複製的設定

兩組設定都來自官方模型卡，只把草稿數改成本次評估要用的值。

122B · NVFP4 + MTP K=3

```
# 目標模型：NVIDIA 量化版，單卡即可
vllm serve nvidia/Qwen3.5-122B-A10B-NVFP4 \
  --quantization modelopt_fp4 \
  --kv-cache-dtype fp8 \
  --tensor-parallel-size 1 \
  --max-model-len 65536 \
  --reasoning-parser qwen3 \
  --enable-auto-tool-choice \
  --tool-call-parser qwen3_coder \
  --speculative-config '{{
    "method": "qwen3_next_mtp",
    "num_speculative_tokens": 3
  }}'

# 官方模型卡建議 2-4；本次取 3
# 容器：nvcr.io/nvidia/vllm:26.04-py3
```

27B · FP8 + DFLASH2 K=7

```
# 目標模型：Qwen 官方 FP8 版
vllm serve Qwen/Qwen3.8-27B-FP8 \
  --max-model-len 65536 \
  --kv-cache-dtype auto \
  --reasoning-parser qwen3 \
  --speculative-config '{{
    "method": "dflash",
    "model": "incoai/Qwen3.8-27B-DFlash2",
    "num_speculative_tokens": 7
  }}'

# block size 8 → 7 個草稿 + 1 個 anchor
# SGLang 對應：--speculative-algorithm DFLASH
#               --speculative-num-draft-tokens 8
```

執行前務必先跑一次 `vllm serve --help` 與 `vllm bench serve --help` 核對你安裝的版本的旗標名稱，[speculative decoding](#) 的介面最近幾個版本改動頻繁。DFlash 在 [vLLM](#) 走 `Speculators` 函式庫，請確認版本支援。

CHAPTER 08

延遲指標的定義與陷阱

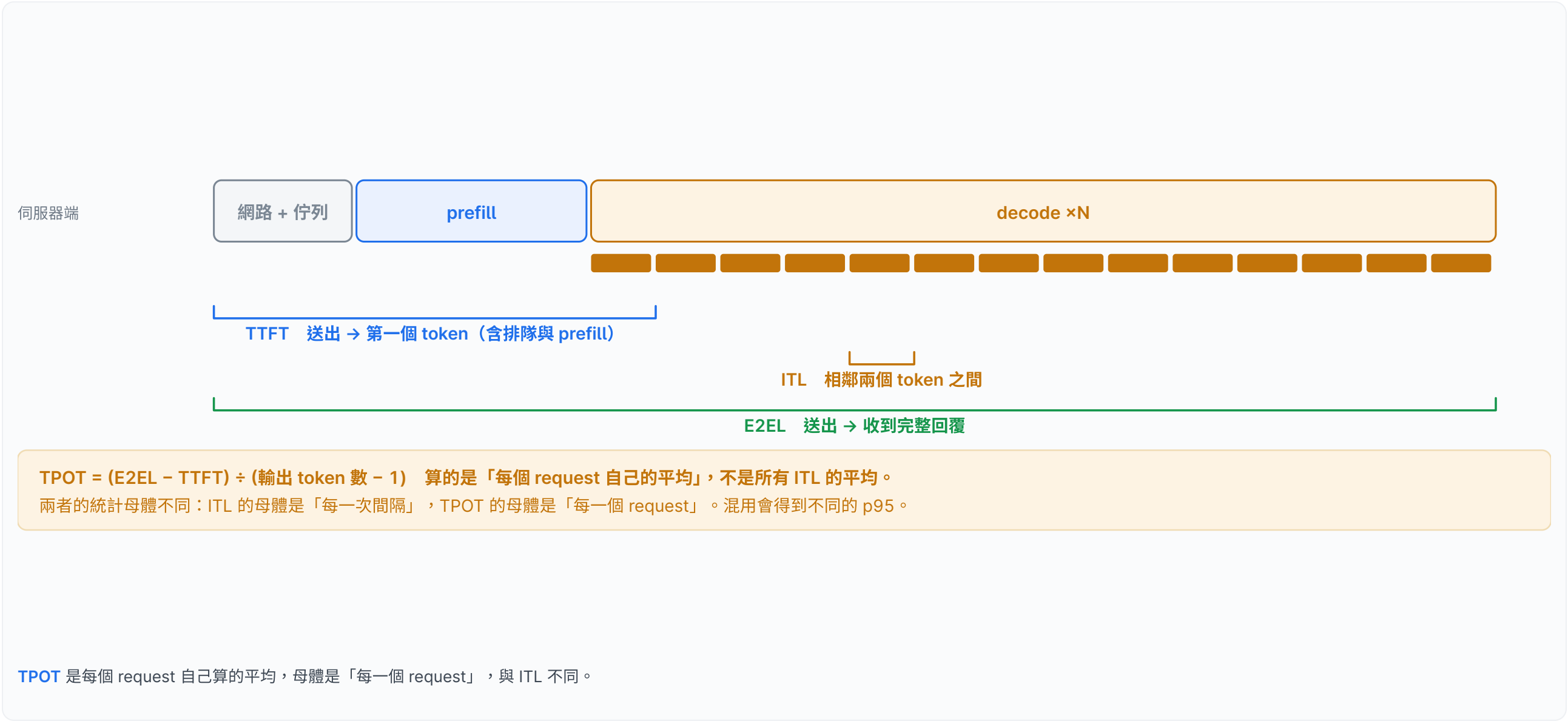
容量數字會被吵，多半不是量錯。是兩邊講的指標、百分位或量測窗根本不一樣。這一章把定義釘死。

- 一個 request 的完整時間軸
- TTFT / TPOT / ITL / E2EL 的精確定義與它們的統計母體
- 為什麼平均值沒有用：百分位與長尾
- Throughput 與 goodput 的差別

一個 request 的完整時間軸

四個指標各自量的是哪一段

TTFT 量的是「等多久才開始」，TPOT / ITL 量的是「開始之後順不順」，E2EL 是使用者實際等待的總時間。



四個指標的精確定義

以及各自的陷阱

每個指標都要問三件事：量的是哪一段、母體是什麼、有沒有含服務外部的時間。

指標	定義	統計母體	常見陷阱
<u>TTFT</u>	送出 request → 收到第一個 token	每個 request 一個樣本	包含排隊時間。系統過載時 <u>TTFT</u> 會先爆炸，而 <u>TPOT</u> 看起來還好。只看 TPOT 會誤判系統健康。
<u>TPOT</u>	$(\text{E2EL} - \text{TTFT}) \div (\text{輸出 token 數} - 1)$	每個 request 一個樣本	只有一個輸出 token 的 request 沒有 TPOT。短輸出佔比高時，樣本會偏少。
<u>ITL</u>	相鄰兩次串流輸出的時間差	每一次間隔一個樣本	開了 <u>speculative decoding</u> 之後，一輪可能吐好幾個 token， <u>ITL</u> 分布變成雙峰，p99 會惡化但平均改善。
<u>E2EL</u>	送出 → 收到完整回覆	每個 request 一個樣本	與輸出長度強相關。比較不同 workload 的 E2EL 沒有意義。

串流與非串流
非串流 API 量不到 TTFT 與 ITL，只有 E2EL。要量延遲細節就必須用串流端點。

客戶端 VS 伺服器端
壓測工具量的是客戶端時間，含網路與 tokenizer。引擎的 metrics 量的是伺服器內部。兩者會差幾十毫秒，報告要說清楚用哪一個。

量測窗
暖機期（CUDA graph 捕捉、cache 預熱）必須排除。建議跑 3-5 分鐘，取中間穩定的 60-120 秒。

高併發時，每個指標各自怎麼動

同一台機器，128 個人同時上門

四個指標不會一起變好或一起變壞。併發一上來，它們會往不同方向跑，而且方向是可以推導的。

指標	併發上升時	為什麼	先撐不住的是
TTFT	急速惡化	排隊時間 = 前面還有幾個人 ÷ 完成率。超過 max_num_seqs 之後，多出來的人只能等，而且等待時間隨排隊人數線性成長。	它。系統過載時 TTFT 一定最先爆
ITL / TPOT	先平、後線性上升	每個 step 的 token 數 = running × (k+1)。低於 N* ≈ 292 時 step 時間不變，所以 ITL 是平的；越過之後 step 時間開始隨 token 數線性成長。	越過屋頂線轉折點之後
E2EL	跟著 TTFT 走	E2E = TTFT + 輸出長度 × TPOT。輸出長時 TPOT 那項主導，輸出短時 TTFT 主導。	看輸出長度
整機 tok/s	上升後飽和	併發越高，同一次權重讀取被越多 token 分攤。到轉折點之後就飽和，再加也不會更多。	飽和之後只剩延遲在漲
Goodput	先升後崩	throughput 還在漲的時候，可能已經沒有人滿足 SLO 了。過載時 goodput 會直接歸零。	這才是真正要盯的那條

排序記起來：TTFT 最先壞、ITL 其次、throughput 最後才飽和、goodput 會突然歸零。只看 throughput 會讓你以為系統還很健康。

數字長什麼樣（27B FP8、IN 2048 / OUT 512、128 人同時上門）							
MAX_NUM_SEQS	跑 / 排隊	TOK/STEP	瓶頸	ITL	TTFT	整機 TOK/S	
16	16 / 112	80	頻寬	3.50 ms	13.43 s	4,576	
32	32 / 96	160	頻寬	3.50 ms	5.59 s	9,152	
48	48 / 80	240	頻寬	3.50 ms	3.08 s	13,728	
64	64 / 64	320	算力	3.83 ms	2.02 s	16,718	
96	96 / 32	480	算力	5.74 ms	1.02 s	16,718	
128	128 / 0	640	算力	7.66 ms	31 ms	16,718	

兩個轉折點

48 → 64 之間，tok/step 從 240 跨到 320，越過 N* = 292。ITL 從這裡開始爬。

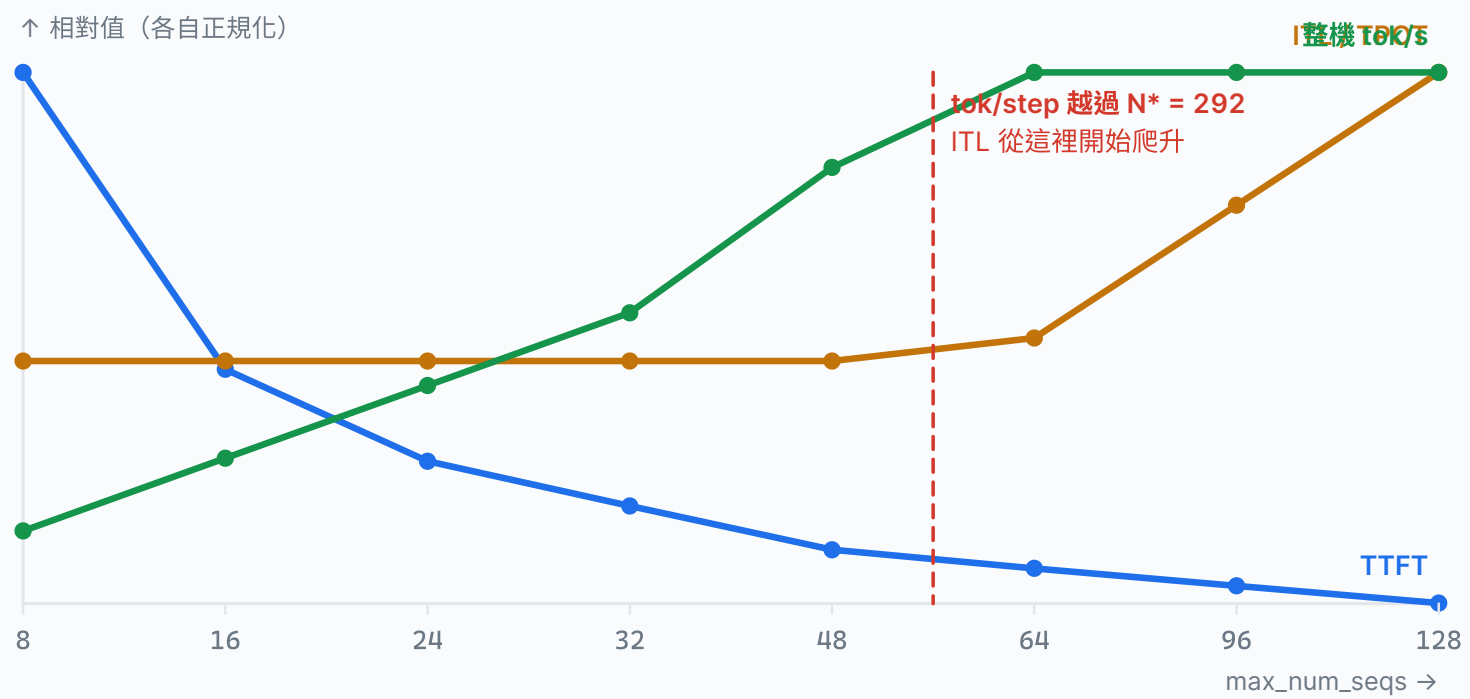
96 → 128 之間，排隊的人歸零。TTFT 從 1 秒掉到 31 ms，但 ITL 已經漲到 7.66 ms。

兩個轉折的位置都可以事先算，不用試。

max_num_seqs 該掐多少

128 個人上門，48 對 96 的差別

掐小它，唯一真正變好的是 ITL；代價是更多人在門外排隊，TTFT、E2E 與吞吐全部變差。



128 個人同時上門		
in 2048 / out 512 , 27B FP8		
	48	96
running / 排隊	48 / 80	96 / 32
tok / step	240	480
瓶頸	頻寬	算力
ITL / TPOT	3.50 ms	5.74 ms
TTFT	3.08 s	1.02 s
E2E	4.87 s	3.96 s
整機 tok/s	13,728	16,718

把 max_num_seqs 掐小，唯一真正變好的是 ITL/TPOT，因為每個 step 的 token 數還停在屋頂線左邊。
代價是 80 個人在門外排隊，TTFT 從 1.0 秒變成 3.1 秒，E2E 與整機吞吐也一起變差。

所以掐小 max_num_seqs 是一筆交易，不是最佳化。

自己模擬一次高併發

拉人數與 max_num_seqs，看四個指標往哪邊跑

同一個模型、同一組 workload，只改 max_num_seqs，就能看出 TTFT 與 ITL 是一組蹺蹺板。

模型

27B FP8122B NVFP432B BF16

speculative

關開

同時上門的人數128

max_num_seqs48

平均輸入長度2,048

平均輸出長度512

48 / 128

同時在跑 / 上門人數 排隊 80 人

240 tok/step

瓶頸在頻寬 (N* ≈ 292)

3.5 ms

ITL / TPOT：使用者看到的打字速度

3.08s

TTFT = 排隊 3,046 ms + prefill 37 ms

4.87 s

E2E：使用者實際等待的總時間

13,728 tok/s

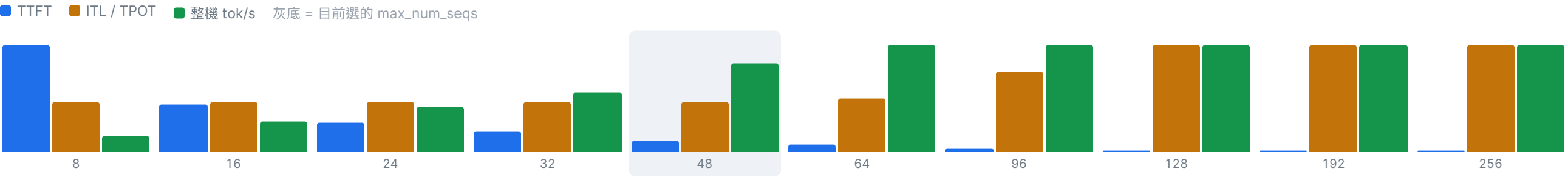
整機輸出（解析上限）

26.27 req/s

穩定態完成率

100%

頻寬佔用（另一側是算力 82%）

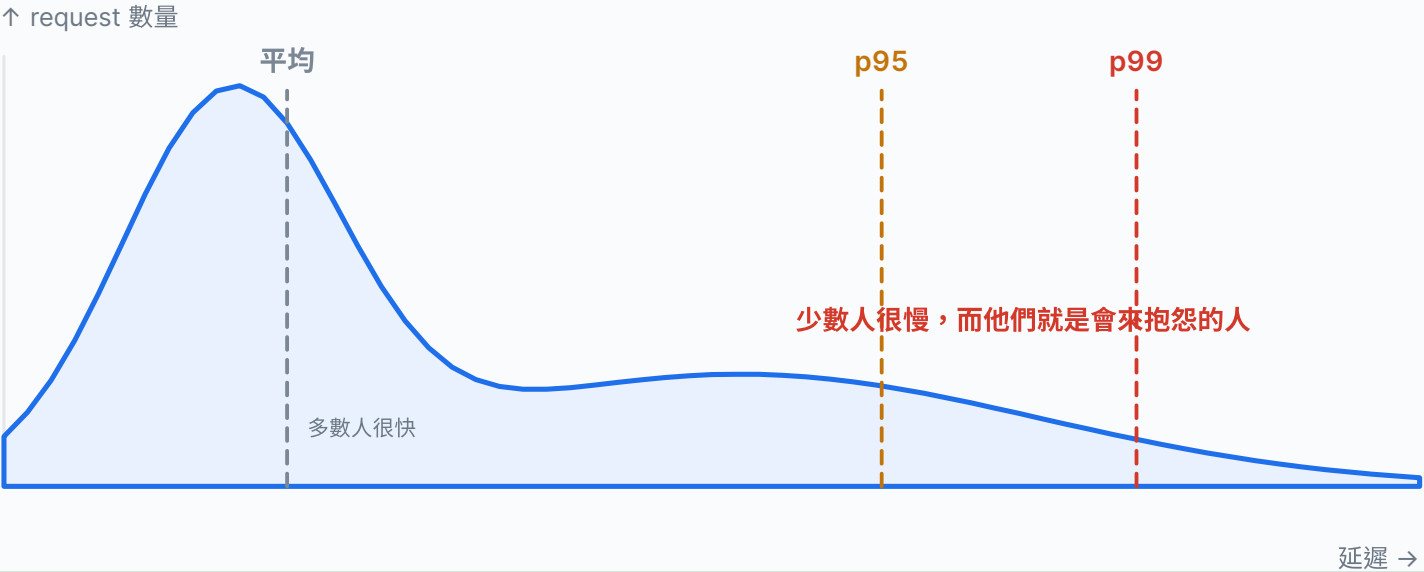


封閉式穩定態近似：running = min(上門人數, max_num_seqs)，其餘排隊；每個 step 撥給 prefill 的 token 數由「進來的人 = 出去的人」決定；step 時間走 roofline。沒有擬合參數，每個數字都可以手算驗證，但也不含 attention kernel、排程與通訊開銷，實測會更慢。

百分位與 goodput

平均值會騙人，goodput 才是使用者拿到的服務

系統過載時 throughput 可能還很高，但沒有一個人滿足 SLO，此時 goodput = 0。



Throughput vs Goodput

throughput：所有輸出 token / 秒

goodput：只算「滿足 SLO」的部分

系統過載時 throughput 可能還很高，
但全部的人都超過 SLO，goodput = 0。

vLLM 的 bench serve 支援

`--goodput ttft:500 tpot:40`

報告紀律：任何一個延遲數字都要寫清楚指標、百分位、量測窗與 workload。

「TPOT 30 ms」沒有意義；「2k in / 512 out、32 併發、穩定 5 分鐘窗內，p95 TPOT = 30 ms」才有。

報告紀律：每個延遲數字都要註明指標、百分位、量測窗與 workload。

CHAPTER 09

兩階段負載測試

「這台機器能服務幾個人」沒有單一答案。它是一個條件式：在這個 workload、這組 SLO、這個引擎版本之下，能承諾多少。這一章給出一套可重現的流程。

- 閉環 vs 開環：為什麼一定要兩階段
- Stage 1：找飽和點（機器的極限）
- Stage 2：找 SLO 容量（可以承諾的量）
- SLO 邊界搜尋的三段式做法
- 從 RPS 換算服務人數：Little's Law 與它的前提
- 報告該寫什麼

閉環 vs 開環

兩種測試回答的是完全不同的問題

閉環量「機器最多能做多少」，開環量「能承諾多少」。只做閉環，你永遠看不到佇列失控的那條線。

Stage 1 閉環飽和測試

- 控制變數：同時併發數（固定 N 個 client，回一個發一個）
- 量測：achieved throughput、TPOT、ITL
- 回答：這台機器最多能做多少
- 特性：佇列不會爆炸，延遲隨併發單調上升

Stage 2 開環 SLO 容量測試

- 控制變數：到達率（Poisson，與伺服器狀態無關）
- 量測：p95/p99 TTFT、TPOT、goodput、佇列長度
- 回答：在 SLO 之下能承諾多少
- 特性：超過容量時延遲以 $1/(1-\rho)$ 爆炸

為什麼不能只做 Stage 1：閉環測試的延遲不會爆炸，因為 client 要等回覆才發下一個，它自帶背壓。真實使用者不會等你，所以只有開環測試才能看到佇列失控的那條線。

為什麼不能只做 Stage 1：閉環自帶背壓，延遲永遠不會爆炸。真實使用者不會等你。

Stage 1：找飽和點

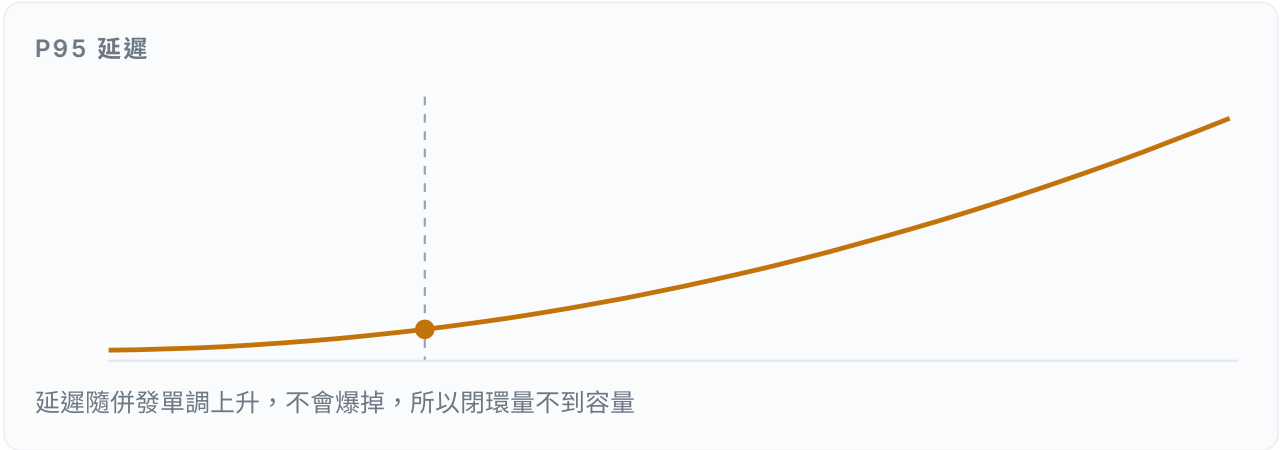
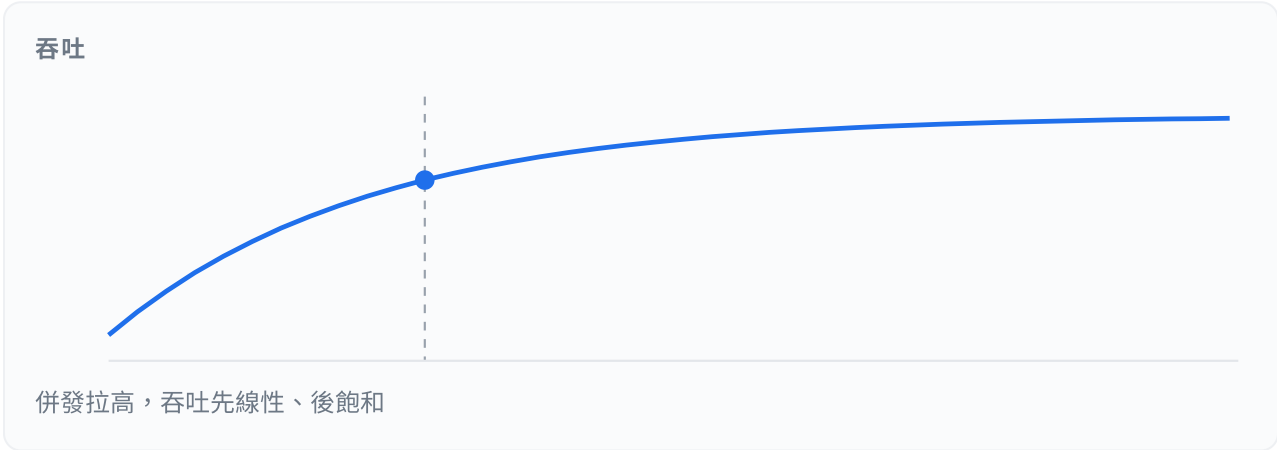
控制併發，量 achieved throughput

併發從 1 開始往上加，看 throughput 什麼時候不再成長，那裡就是這台機器的物理極限。

測試型態

Stage 1 閉環（控併發） Stage 2 開環（控到達率）

控制變數 12



1,031 tok/s
併發 = 12

119ms
p95 ITL

量到飽和點
閉環只能回答「機器最多做多少」

此圖為教學示例曲線（以排隊理論生成），不是任何硬體的實測結果。正式報告必須換成實測資料，並附上引擎版本、workload、量測窗與 SLO 定義。

Stage 2：找 SLO 容量

控制到達率，看佇列與延遲是否穩定

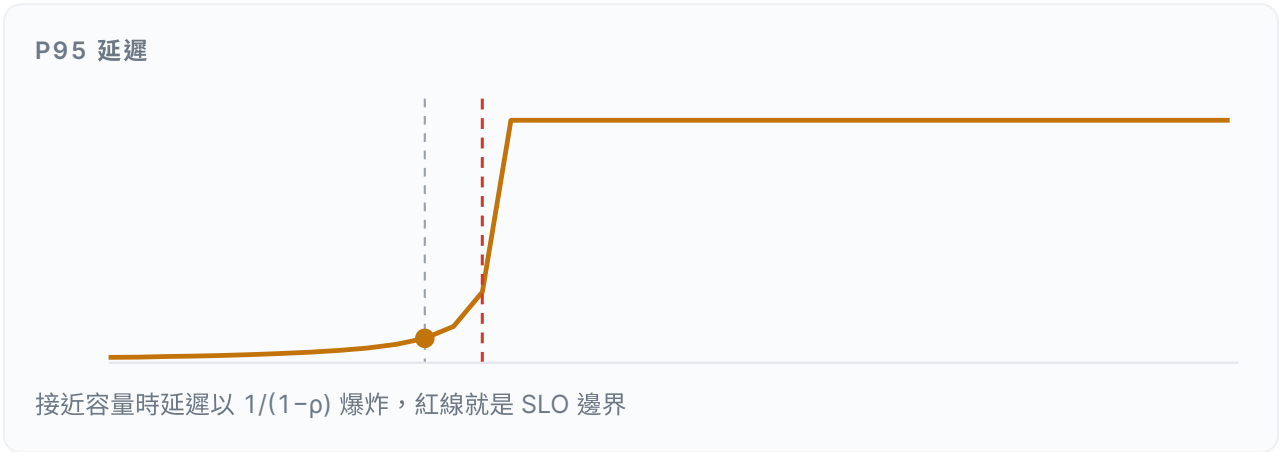
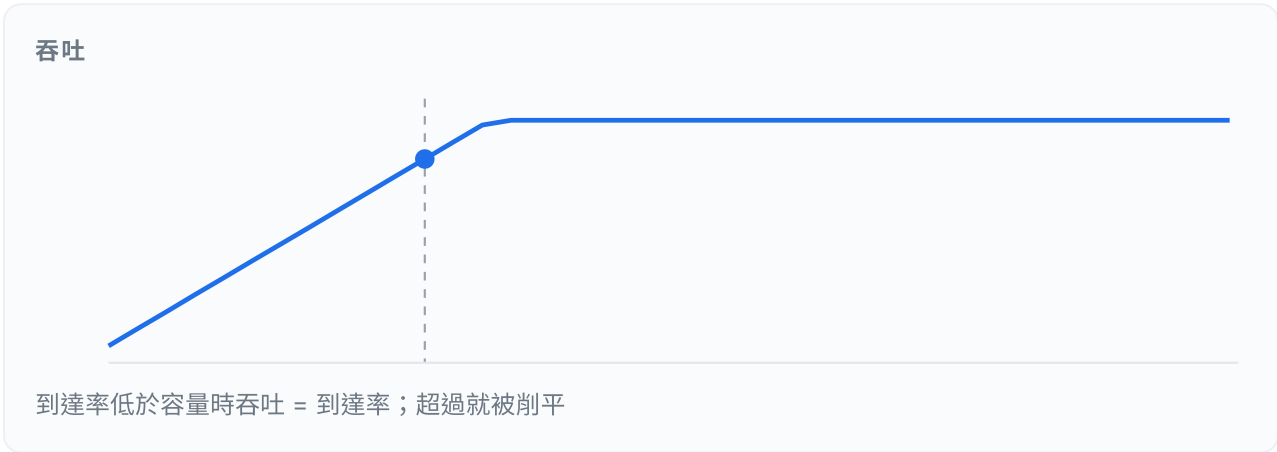
到達率低於容量時，吞吐等於到達率、延遲平穩；一超過，延遲就以 $1/(1-\rho)$ 爆炸。

測試型態

Stage 1 閉環（控併發）

Stage 2 開環（控到達率）

控制變數 12



1,200 tok/s

到達率 = 12 req/s

222 ms

p95 TTFT

SLO 通過

開環才能回答「能承諾多少」

此圖為教學示例曲線（以排隊理論生成），不是任何硬體的實測結果。正式報告必須換成實測資料，並附上引擎版本、workload、量測窗與 SLO 定義。

- 穩定的判準（三個都要成立）
- 佇列長度沒有往上的趨勢
 - throughput $\approx$ 到達率，沒有被削平
 - p95 延遲在量測窗內平穩，不隨時間漂移

每個點要跑多久

暖機 60 秒 + 量測 120 秒是常見起點。判斷是否穩定：把量測窗切成前後兩半，如果 p95 差超過 10%，代表還沒穩定。

到達率必須是 Poisson（或真實 trace）而不是等間隔。等間隔會低估佇列效應，讓容量估計偏樂觀。

SLO 邊界搜尋

粗掃描範圍，細掃描逼近，再重複確認

SLO 容量是一個邊界值，要用搜尋找出來，而且必須驗證可重現。



三段式搜尋

- ① 粗掃
大步長找出崩潰的區間
- ② 細掃
在區間內二分逼近
- ③ 重複確認
邊界點跑 3 次以上
確認可重現、無趨勢

每個點都要跑到穩定態

教學示例曲線（以 M/M/1 近似生成），不是實測結果。真實曲線的形狀相同，但轉折位置完全取決於你的模型、硬體與 workload。

③ 重複確認：邊界點至少跑 3 次，確認可重現、量測窗內沒有趨勢。

從 RPS 換算服務人數

Little's Law 與它的四個前提

$\lambda = \text{goodput} \div \text{平均輸出長度}$ ；人數 = $\lambda \div \text{每人的到達率}$ 。只有在系統穩定時才成立。



11.72req/s

$\lambda = \text{goodput} \div \text{平均輸出長度}$

257.8

$L = \lambda \times W$ ：平均同時在系統內的 request 數
→ 對照 max_num_seqs 是否夠

180s

每位使用者平均隔多久發一次

2,109

可服務人數 (每人 20 輪/小時)

$\lambda = 6,000 \div 512 = 11.72 \text{ req/s}$ · $L = \lambda \cdot W = 257.8$ · 人數 = $\lambda \div (20/3600) = 2,109$

前提：系統穩定（到達率 < 服務率、佇列不成長）、輸出長度分布與量測窗一致、在該吞吐下 SLO 仍然滿足。任何一項不成立，這個人數就不能引用。注意 goodput 要用「通過 SLO」的部分，不是原始 throughput。

報告該寫什麼

一份可以被別人重跑的容量報告

容量數字沒有附上這張表的内容，就無法被驗證，也無法被比較。

① 測試設定

引擎	<u>vLLM</u> 版本、容器 image、啟動旗標全文
模型	repo 名稱 + revision hash 、量化方式
起草器	repo + revision、method、num_speculative_tokens
硬體	GPU 型號、數量、TP/EP/DP 設定、驅動版本
快取	kv-cache-dtype、max-model-len、max-num-seqs、max-num-batched-tokens、gpu-memory-utilization、 <u>prefix caching</u> 開關

③ SLO 與量測

SLO	指標 + <u>百分位</u> + 門檻，例如「p95 <u>TTFT</u> ≤ 800 ms 且 p95 <u>TPOT</u> ≤ 40 ms」
量測窗	暖機時間、量測長度、重複次數
穩定性證據	佇列長度時間序列、前後半窗的 p95 差異

② WORKLOAD

輸入長度	分布（不只是平均）：p50 / p95 / max
輸出長度	同上；是否設 max_tokens 上限
前綴重複度	prefix cache 命中率（開 spec 前後都要量）
思考模式	thinking 開／關，會大幅改變輸出長度
到達模式	Poisson / 真實 trace / 等間隔

④ 結果

Stage 1	飽和併發數、峰值 throughput、當時的 TPOT
Stage 2	SLO 容量（req/s）、goodput、邊界點的重複結果
換算	人數 + 使用者行為假設 + Little's Law 驗算
對照組	開／關 speculative decoding、不同 k 的同條件比較

濃縮成一行：「容量」 = f(模型版本, 量化, 引擎設定, workload, SLO, 硬體)。少寫任何一個自變數，那個數字就不能被引用。

CHAPTER 10

可以帶走的十個結論

以及一張「該轉哪個旋鈕」的決策圖，跟一份誠實的「這份報告沒有回答什麼」。

- 五個關於機制的結論
- 五個關於設定與量測的結論
- 從症狀出發的調校決策圖
- 必須自己量的六件事

十個結論

把整份報告壓成十句話

機制

- 1. **decode 是 memory-bound**。一次 forward 讀完整份權重，只服務 B 個 token。所有最佳化都在提高「每次讀取分攤到的 token 數」。
- 2. **B200 的屋頂線轉折點是 $N^* \approx 292$ token/step，而且與精度無關**。量化讓兩邊同時變快，不會改變你的相對位置。
- 3. **混合架構把 KV cache 砍掉 3/4，但換來固定的 SSM state**。122B 的 KV 只有 12 KiB/token (**FP8**)，Qwen3-32B 是 256 KiB。
- 4. **SSM state 是每序列 ~147 MiB 的固定成本**。短對話時它**主導**記憶體：122B 要到 12,720 token KV 才追上它。
- 5. **MoE 省算力不省記憶體，而且 batch 一大就要讀滿所有 expert**。122B 從 batch 1 的 12.81 GB 長到高併發的 76.00 GB。

設定與量測

- 6. **NVFP4 只量化了 routed expert**。36 層 GDN、12 層 attention、lm_head、MTP head 全部還是 BF16，佔了 checkpoint 的 21.9% 與 batch-1 頻寬的一半。
- 7. **NVFP4 讓 122B 單卡跑得動** (83.47 GB vs BF16 的 250.17 GB)，可以用 DP 取代 TP，省掉 all-reduce。
- 8. **speculative decoding 的效益由 $B \times (k+1)$ 是否超過 N^* 決定**。27B 配 DFlash2 k=7 → 安全併發約 36；超過就開始賠。
- 9. **接受率不是常數**。context 從 2k 長到 30k，實測平均接受率從 65% 掉到 39%，吞吐從 +129% 變成 -51%。長 context 一定要重量。
- 10. **容量是一個條件式，不是一個數字**。f(模型版本, 量化, 引擎設定, workload, SLO, 硬體)。少一個自變數就不能引用。

該轉哪個旋鈕

從症狀出發的決策圖

先看指標，再動旗標。一次只動一個，每次用同一組 workload 與 SLO 重測。



下一步：這份報告沒有回答的問題

誠實列出邊界

這份報告給的是機制、公式與可驗證的架構數字。實測數字必須自己跑。

必須自己量的

- 兩個模型在你的 workload 上的**實際** throughput / TTFT / TPOT 曲線
- speculative decoding 的**實際**逐位置接受率與 τ
- 開／關 spec 的 goodput 對照（同一組 SLO）
- prefix cache 命中率，以及開 spec 之後的變化
- 量化對**你的**驗收集的影響（不是官方 benchmark）
- 長 context (> 32k) 的接受率與吞吐

這份報告刻意沒有涵蓋的

- 訓練與微調
- 多模態（視覺塔）的容量影響，只在參數帳本裡列出大小
- P/D 分離（prefill-decode disaggregation）
- 多節點部署與網路拓撲
- 成本模型（\$/百萬 token）
- 模型品質的橫向評比

建議的下一步：用 CH9 的兩階段流程，對兩個模型各跑一次基準測試，並把結果填回 CH6 與 CH7 的試算器，驗證解析模型與實測的差距有多大。解析上限通常是實測的 1.7–3 倍；知道自己的比例之後，以後就能用試算器快速估算新設定。

APPENDIX

附錄

給 Q&A 與後續查證用。

- A 名詞表（也可以隨時按 G 打開）
- B config 與 checkpoint 的原始證據
- C 可以直接複製的命令
- D GDN 的數學細節
- E 單卡與多卡：TP / EP / DP
- F 來源清單與資料界線

附錄 A：名詞表（1/2）

Acceptance length (τ) – HBM · 共 41 條；隨時按 G 也能打開

Acceptance length (τ)

speculative decoding 每一輪平均被接受的 token 數（含最後那個「保底」token）。 τ 越大越省。k 個草稿的上限是 k+1。

Activated parameters

MoE 模型每產生一個 token 真正參與運算的參數量。Qwen3.5-122B-A10B 的 122B 是總量，A10B 指每個 token 只用約 10B。算力照 10B 算，記憶體要放滿 122B。

Arithmetic intensity

每從記憶體讀 1 byte 能做幾次浮點運算。prefill 很高（吃算力），decode 很低（吃頻寬）。報告裡談的最佳化，多半都是在拉高這個數字。

Autoregressive

一次只產生一個 token，再把它接回輸入產生下一個。生成 T 個 token 就要跑 T 次 forward，decode 慢的原因就在這裡。

BF16

bfloat16，16 位元浮點。與 FP16 同寬但指數位更多，訓練與推論的預設精度。1 個參數佔 2 bytes。

Block diffusion

DFlash2 用的起草方式：把未來一整段位置全部遮起來，用一次 forward 同時「去噪」猜出整塊 token，而不是一個一個猜。

Chunked prefill

把很長的 prompt 切成小塊，分散到多個排程輪次，避免一個長 prompt 讓其他人的 decode 卡住。

Continuous batching

每一輪排程都重新組合正在跑的 requests：做完的離開、等待的立刻補進來，不用等整個 batch 一起結束。

CUDA graph

把一串固定的 GPU kernel 呼叫錄下來重播，省掉每次啟動 kernel 的 CPU 開銷。decode 步驟很短，這個開銷佔比很高。

Decode

生成階段。每一輪每條序列只餵 1 個新 token，讀取整份權重與全部歷史 KV，屬於 memory-bound。

DFlash2

z-lab / IncoAI 的 block-diffusion 起草模型。用非因果 attention 同時看目標模型的 hidden state 與 mask token，一次 forward 產生整塊草稿。本報告 27B 使用 block size 8、7 個草稿 token。

E2EL

End-to-end latency，從送出 request 到收到完整回覆的總時間。

EAGLE / EAGLE-3

以目標模型的 hidden state 為條件的輕量起草器。EAGLE-3 融合目標模型低／中／高層特徵，並在訓練時模擬多步生成。Qwen 的 MTP head 屬於同一系。

Expert parallelism (EP)

把 MoE 的專家分散到不同 GPU。省記憶體，但每一層都要做 all-to-all 把 token 送到專家所在的卡。

FP8 (E4M3)

8 位元浮點，4 位指數 3 位小數。1 個參數 1 byte。Qwen3.8-27B-FP8 用 128 為單位的 blockwise 縮放。

Full attention

標準的 softmax attention，需要保存整段歷史的 K/V。Qwen 混合架構每 4 層放 1 層。

Gated Attention

Qwen3.5 系列的 attention 變體，輸出乘上一個學出來的 gate（config 的 attn_output_gate=true），所以 q_proj 的輸出寬度是兩倍。

Gated DeltaNet (GDN)

一種 linear attention。用固定大小的矩陣狀態 S 取代 KV cache，靠 decay gate α 與 delta rule β 更新。記憶體不隨 context 成長。

Goodput

只計算「滿足 SLO」的產出速率。相對於 throughput 把所有輸出都算進去，goodput 才反映使用者真正拿到的服務。

GQA

Grouped-Query Attention。多個 Q head 共用一組 K/V head，直接把 KV cache 縮小成 1/(head 比例)。

HBM

GPU 上的高頻寬記憶體。B200 單卡 180 GB、約 7.7 TB/s。權重、KV cache、SSM state、activation 全部共用這塊。

附錄 A：名詞表（2/2）

ITL – vLLM · 共 41 條；隨時按  也能打開

ITL
Inter-token latency，串流時相鄰兩次輸出之間的間隔。使用者感受到的「順不順」。

KV cache
把每個位置算過的 K/V 存起來，避免每輪重算。大小 = 2 × full-attention 層數 × KV head 數 × head_dim × 位元組數 × token 數，隨 context 線性成長。

Little's Law
 $L = \lambda \times W$ 。穩定系統裡，平均同時在系統內的 request 數 = 到達率 × 平均停留時間。把 RPS 換算成「同時服務幾個人」的依據。

MoE
Mixture of Experts。把 FFN 換成很多個小 expert，每個 token 只選幾個來算。省算力，不省記憶體。

MTP
Multi-Token Prediction。模型自帶的額外一層 head，訓練時就學著預測後面幾個 token，可直接當 **speculative decoding** 的起草器。Qwen3.5/3.8 的 checkpoint 內含 mtp.* 權重。

NVFP4
NVIDIA Blackwell 的 4 位元浮點格式。每 16 個 E2M1 值共用一個 FP8-E4M3 的 block scale，再乘上一個 per-tensor 的 FP32 scale，平均 4.5 bit／值。

PagedAttention
把 KV cache 切成固定大小的 block，用 block table 把邏輯位置對應到不連續的實體 block，像作業系統的分頁一樣，消除記憶體碎片。

Percentile (p95/p99)
延遲分布的第 95／99 百分位。平均值會被少數快的請求拉低，SLO 必須看百分位。

Prefill
把整段 prompt 一次算完並填好 KV cache 的階段。token 多、算術強度高，屬於 compute-bound。

Prefix caching
不同 request 共用的開頭（system prompt、few-shot）只算一次，KV block 直接重用。

Rejection sampling
speculative decoding 的驗證方式：逐位置決定接受或拒絕草稿，數學上讓最終輸出分布與不猜時完全一致。

Roofline
把「頻寬上限」與「算力上限」畫在一起的模型。兩條線交會處就是轉折點，決定你在 memory-bound 還是 compute-bound。

RoPE
Rotary Position Embedding。把位置資訊以旋轉的方式寫進 Q/K。Qwen3.5 只旋轉 head_dim 的 1/4（partial_rotary_factor 0.25）。

SLO
Service Level Objective。事先講定的延遲目標，例如「p95 TTFT < 800 ms 且 p95 TPOT < 40 ms」。沒講定 SLO，容量數字就沒有意義。

Speculative decoding
先用便宜的方式猜 k 個 token，再讓目標模型一次驗證。猜對就一次前進多格，猜錯就丟掉。輸出分布不會變。

SSM cache / recurrent state
GDN 等 linear attention 層要保存的狀態：一個 $d_v \times d_k$ 的矩陣（FP32）加一小段 convolution 歷史。大小固定，與 context 長度無關。

Tensor parallelism (TP)
把每一層的矩陣切開放在多張卡上，每層都要做一次 all-reduce。降低單卡記憶體與延遲，代價是通訊。

TPOT
Time Per Output Token，第一個 token 之後的平均每 token 時間。等於「使用者看到的打字速度」的倒數。

TTFT
Time To First Token，送出 request 到收到第一個 token 的時間。包含排隊與 prefill。

vLLM
本報告使用的推論伺服器。負責排程、KV cache 分頁管理、continuous batching 與 kernel 呼叫。

附錄 B：config 與 checkpoint 的原始證據

所有架構數字的來源

關鍵 CONFIG 欄位

```
// Qwen3.5-122B-A10B / text_config
"num_hidden_layers": 48,
"full_attention_interval": 4,
"hidden_size": 3072,
"num_attention_heads": 32,
"num_key_value_heads": 2,
"head_dim": 256,
"partial_rotary_factor": 0.25,
"attn_output_gate": true,
"linear_num_key_heads": 16,
"linear_num_value_heads": 64,
"linear_key_head_dim": 128,
"linear_value_head_dim": 128,
"linear_conv_kernel_dim": 4,
"mamba_ssm_dtype": "float32",
"num_experts": 256,
"num_experts_per_tok": 8,
"moe_intermediate_size": 1024,
"shared_expert_intermediate_size": 1024,
"mtp_num_hidden_layers": 1,
"vocab_size": 248320,
"max_position_embeddings": 262144
```

量化設定與 DTYPE 統計

```
// nvidia/...-NVFP4 hf_quant_config.json
"quant_algo": "NVFP4",
"kv_cache_quant_algo": "FP8",
"group_size": 16

// safetensors dtype 統計 (HF API)
122B-NVFP4  BF16    9,122,380,016
              F8_E4M3 7,247,757,312
              U8      57,982,058,496
              total_size 83,474,870,752 B

27B-FP8      BF16    3,082,220,272
              F8_E4M3 24,699,207,680
              weight files 30,866,866,928 B

// Qwen3.8-27B / text_config 的差異
"num_hidden_layers": 64 // 48 GDN + 16 full
"hidden_size": 5120
"num_key_value_heads": 4
"linear_num_value_heads": 48
"intermediate_size": 17408 // dense

// incoai/Qwen3.8-27B-DFlash2
"dflash_config": { "block_size": 8,
                  "selector_rank": 256, "selector_top_k": 16,
                  "target_layer_ids": [5,19,33,47,61] }
```

附錄 C：可以直接複製的命令

服務啟動與壓測

STAGE 1：閉環飽和測試

```
vllm bench serve \  
  --backend openai-chat \  
  --base-url http://HOST:8000 \  
  --endpoint /v1/chat/completions \  
  --model MODEL_NAME \  
  --dataset-name random \  
  --random-input-len 2048 \  
  --random-output-len 512 \  
  --num-prompts 600 \  
  --max-concurrency 32 \  
  --percentile-metrics ttft,tpot,itl,e2el \  
  --metric-percentiles 50,95,99 \  
  --save-result --result-filename c32.json  
  
# 併發掃描：1 2 4 8 16 32 64 128  
# 記錄每一點的 throughput 與 p95 TPOT
```

STAGE 2：開環 SLO 容量測試

```
vllm bench serve \  
  --backend openai-chat \  
  --base-url http://HOST:8000 \  
  --endpoint /v1/chat/completions \  
  --model MODEL_NAME \  
  --dataset-name random \  
  --random-input-len 2048 \  
  --random-output-len 512 \  
  --num-prompts 1200 \  
  --request-rate 12 \  
  --burstiness 1.0 \  
  --goodput ttft:800 tpot:40 \  
  --percentile-metrics ttft,tpot,itl,e2el \  
  --metric-percentiles 50,95,99 \  
  --save-result --result-filename r12.json  
  
# burstiness 1.0 = Poisson  
# 到達率掃描：粗掃 → 細掃 → 邊界重複 3 次
```

執行前一定要跑 `vllm bench serve --help` 核對你安裝的版本 的旗標名稱。另外：每一輪之間若要公平比較，記得重啟服務或清空 prefix cache，否則後面的測試點會因為快取命中而虛胖。

附錄 D：GDN 的數學細節

單一 head 的更新式

$$S_t = S_{t-1} (\alpha_t (I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T$$

$$o_t = S_t q_t$$

- $S_t \in \mathbb{R}^{d_v \times d_k}$ ，每個 value head 一份
- $\alpha_t \in (0,1)$ ：decay gate，由 `in_proj_a` 與 `A_log`、`dt_bias` 產生
- $\beta_t \in (0,1)$ ：delta rule 強度，由 `in_proj_b` 產生
- $q、k、v$ 由 `in_proj_qkv` 一次產生，再經短 convolution（`conv1d`，kernel = 4）
- 輸出還會乘上 `in_proj_z` 產生的 output gate，並過 `norm` 之後由 `out_proj` 投影回 hidden
- 此處省略 normalization 與 head 之間的細節

VLLM 的 STATE SHAPE

```
conv_dim = d_k·n_k·2 + d_v·n_v

conv_state_shape = (
    conv_dim / tp,
    conv_kernel - 1 + num_spec,
)

temporal_state_shape = (
    n_v / tp, d_v, d_k,
)

# 122B: conv_dim = 12,288
#       state = (64, 128, 128)
# 27B : conv_dim = 10,240
#       state = (48, 128, 128)
```

為什麼 CHUNKWISE 能跑滿 TENSOR CORE

把 α 的累積乘積提出來之後，gate 只剩下逐元素乘法，不破壞矩陣乘法的結構。論文的說法：

「The gating term only performs elementwise multiplication with intermediate variables without affecting matrix multiply structures.」
Gated Delta Networks, arXiv 2412.06464

附錄 E：單卡與多卡

TP / EP / DP 各自改變什麼

Tensor Parallel (TP)

把每一層的矩陣切開

- 每層都要 all-reduce
- 降低單卡權重與 KV (KV head 夠切時)
- 122B 只有 2 組 KV head → TP 最多切到 2
- 延遲敏感場景常用

Expert Parallel (EP)

把 MoE 的專家分散

- 每層 all-to-all 把 token 送到專家
- 只有 MoE 模型適用
- 大幅降低單卡權重
- 負載不均會放大尾端延遲

Data Parallel (DP)

整份模型複製多份

- 沒有跨卡通訊
- 吞吐線性成長
- 每張卡都要放得下整個模型
- NVFP4 讓 122B 可以這樣做

本次評估的關鍵結論：NVFP4 把 122B 壓到 83.5 GB，可以用 TP=1 單卡跑。
8 卡的節點因此可以跑 8 份獨立的 DP 副本，取代 1 份 TP=8。沒有 all-reduce，也沒有通訊抖動，而且每份副本的 cache 空間完全獨立。缺點是無法服務超長 context（單卡 cache 有限）。

NVFP4 讓 122B 可以走 DP 取代 TP。這算本次評估最重要的工程結論之一。

附錄 F：來源與資料界線

查核日期 2026-09-08

本報告引用的來源（點擊開啟）

- Qwen3.5-122B-A10B config.json
- Qwen3.5-122B-A10B 模型卡
- nvidia/Qwen3.5-122B-A10B-NVFP4 模型卡
- NVFP4 hf_quant_config.json
- NVFP4 safetensors index
- Qwen3.8-27B config.json
- Qwen3.8-27B 模型卡
- Qwen3.8-27B-FP8 模型卡
- Qwen3.8-27B-FP8 safetensors index
- Qwen3-32B config.json
- Qwen3 Technical Report (arXiv 2505.09388)
- Qwen3.8-27B tokenizer.json（本報告用它實跑 BPE）
- Mamba: Linear-Time Sequence Modeling with Selective State Spaces
- NVlabs/GatedDeltaNet 官方實作
- HuggingFace transformers：Qwen3.5 模型文件
- incoai/Qwen3.8-27B-DFlash2 模型卡
- DFlash: Block Diffusion for Flash Speculative Decoding
- NVIDIA：Boost Inference up to 15x with DFlash
- vLLM Speculators：DFlash 演算法文件
- EAGLE-3 (arXiv 2503.01840)
- vLLM Speculative Decoding 文件
- SmartSpec：以 goodput 最佳化 speculative decoding
- An Interpretable Latency Model for Speculative Decoding
- vLLM #47602：MTP 接受率隨 context 衰減
- vLLM #38182：MTP 降低 prefix cache 命中率
- vLLM Architecture Overview
- Inside vLLM: Anatomy of a High-Throughput LLM Inference System
- vLLM PagedAttention 發表文
- vLLM Hybrid KV Cache Manager 設計文件
- vLLM mamba_utils.py（state shape 計算）
- vLLM Qwen GDN linear attention 實作
- vllm bench serve CLI 文件
- vLLM goodput metric（PR #9338）
- Gated Delta Networks (arXiv 2412.06464)
- NVFP4 格式說明（Modal LLM Almanac）
- NVFP4：Blackwell microscaling 4-bit float
- NVIDIA Blackwell / HGX B200 datasheet
- Lenovo ThinkSystem HGX B200 180GB 產品指南（規格表）

實測 / 官方

架構參數、參數量、checkpoint 位元組數、量化設定、**NVFP4** 精度評測、**DFlash2** 的**接受長度**與加速倍數、**vLLM** issue 的實測數字、B200 規格。全部來自上列來源，可直接引用。

公式推導

KV / SSM cache 大小、記憶體帳本、**roofline** 轉折點、每 step 的位元組流量、**speculative decoding** 的加速估計。公式都寫在頁面上。標示「解析上限」者，實測通常是其 30-60%。

教學示例

CH9 的所有 throughput / 延遲 / 佇列曲線與 SLO 掃描圖。以排隊理論生成，**不是任何硬體的實測結果**。正式報告必須替換成自家實測資料。

CHECKPOINT REVISION

nvidia/Qwen3.5-122B-A10B-NVFP4
98915d837c4e7c87ac8296d02e89de19b3207e6d
Qwen/Qwen3.8-27B-FP8
017b9c7af6b5689d5dd426a76e0bc077eb5ca20a
incoai/Qwen3.8-27B-DFlash2
dedf8df68adfb1afeaf7b7480c0a0243108177b4
Qwen/Qwen3.5-122B-A10B
dc4d348443bc740c68e2d77492492c11606384d5
Qwen/Qwen3.8-27B
1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0
Qwen/Qwen3-32B
9216db5781bf21249d130ec9da846c4624c16137

圖片：continuous batching 與 PagedAttention 兩張為使用者提供，出自 vLLM 官方部落格。其餘所有圖表均為本報告自行繪製的 SVG。